

Feed Forward NN

Advanced NLP M26

Manish Shrivastava + Aparajitha



August 27, 2026

Transformer

Where are we?

FFNN

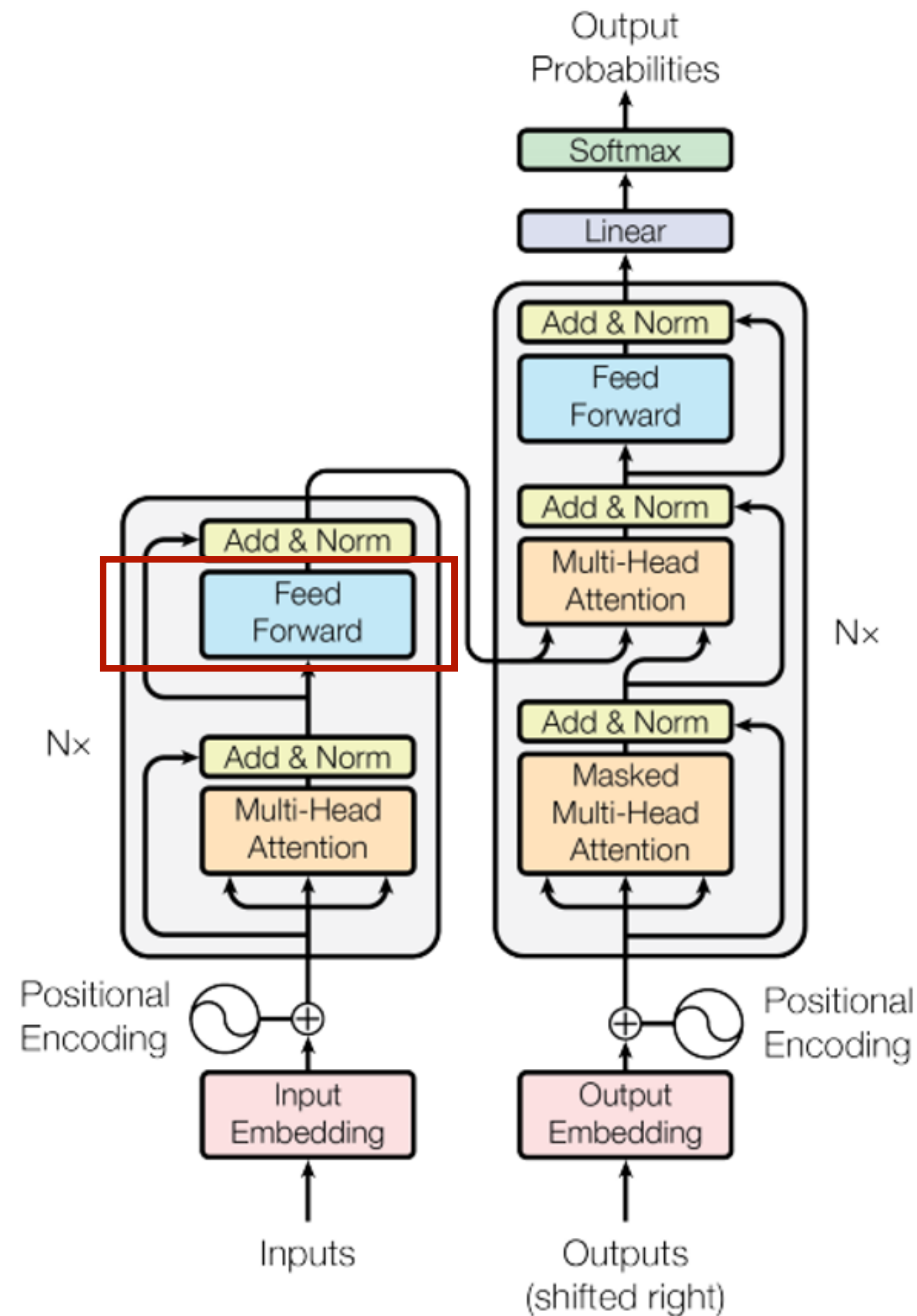


Figure 1: The Transformer - model architecture.

Transformers

Parameter Counts ~1B Model

V	Vocab Size	32000
d	Embedding Dim	2048
n	Context Len	2048
dq	Q K V Dim	128
h, l	Heads, Layers	16

Component	Calculation	~Param	~%
Token Embedding	$V * d$	65M	6.95
Positional Embedding	$n * d$	4.2M	0.44
Attention	$4 * d_q * d * h * l$	268M	28.63
FFNN	$2 * d * 4d * l$	536M	57.37
Decoding	$d * V$	65M	6.95
Bias + Layer Norm		~0.3M	0.03
Total		935.8M	

FFNN

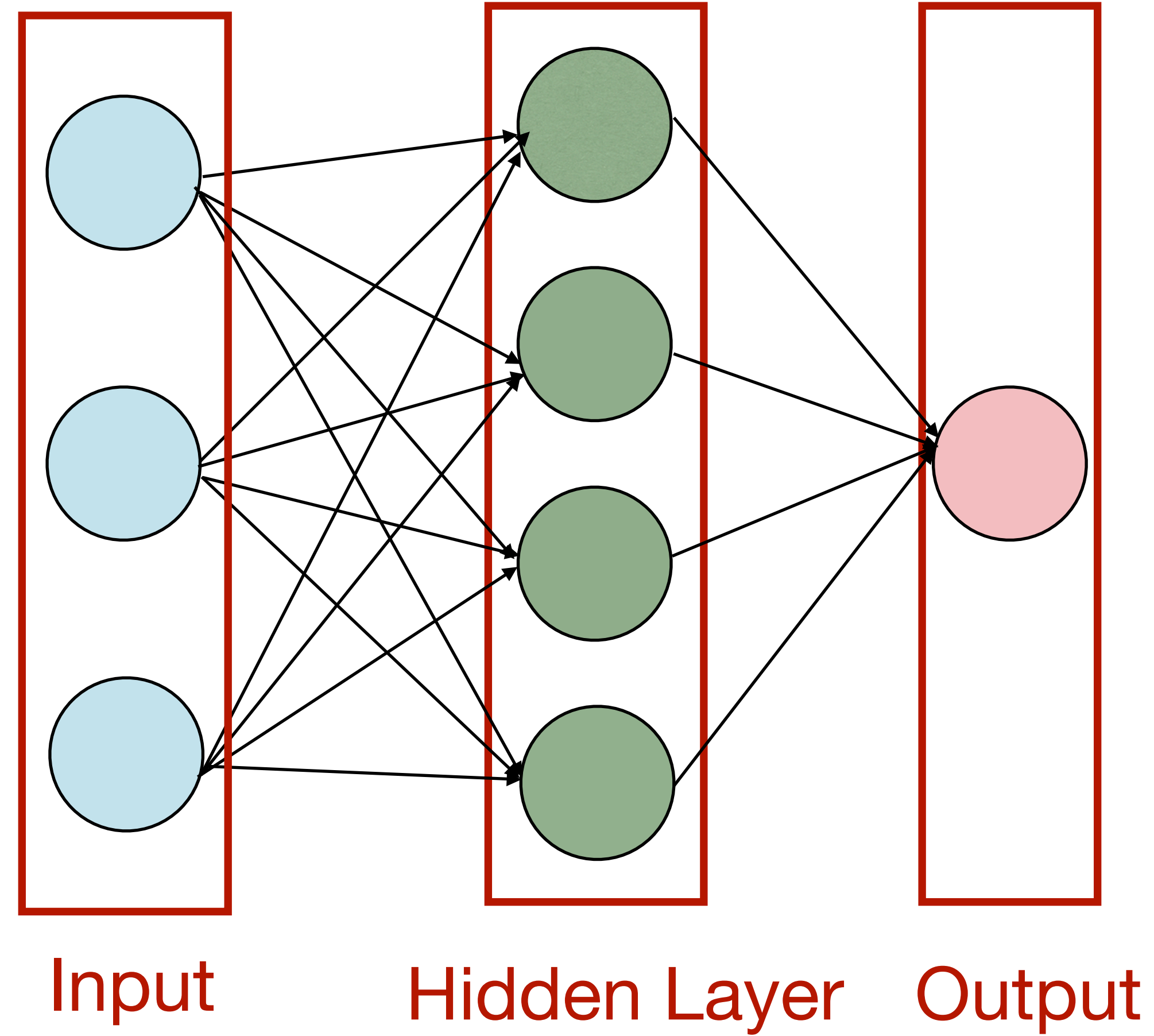
Importance

- Adds non-linearity to the model
- Without this Layer, the stacked attention layers would be linear



FFNN

Recap



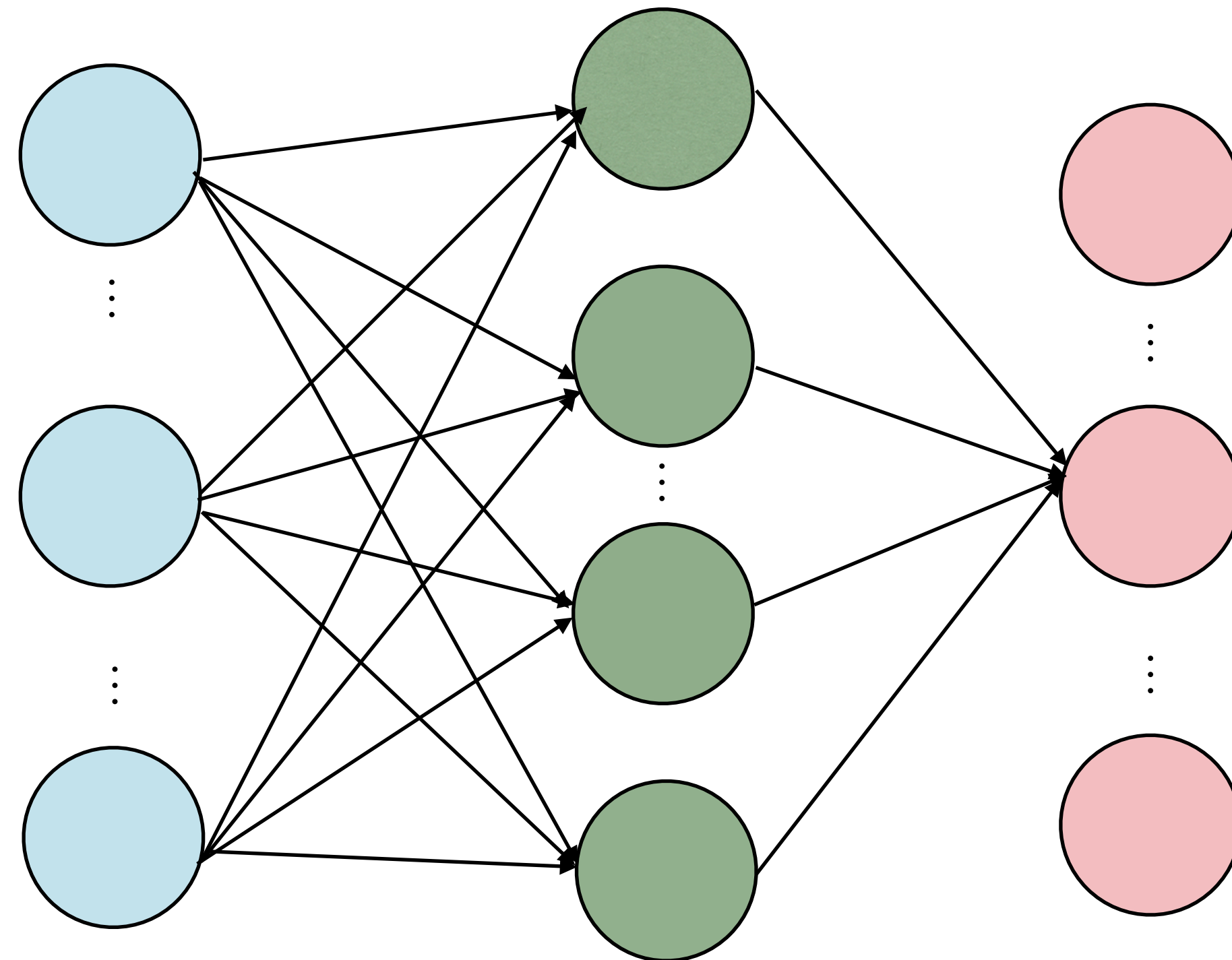
FFNN

Recap

$$f: \mathbb{R}^{D_i} \rightarrow \mathbb{R}^{D_o},$$

$$\mathbf{y} = \mathbf{f}[\mathbf{x}, \phi]$$

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right]$$



$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d$$



Activation Function

$$\mathbf{y} = \mathbf{f}[\mathbf{x}, \phi] \quad f: \mathbb{R}^{D_i} \rightarrow \mathbb{R}^{D_o},$$

$$h_d = \boxed{a} \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right]$$

$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d$$



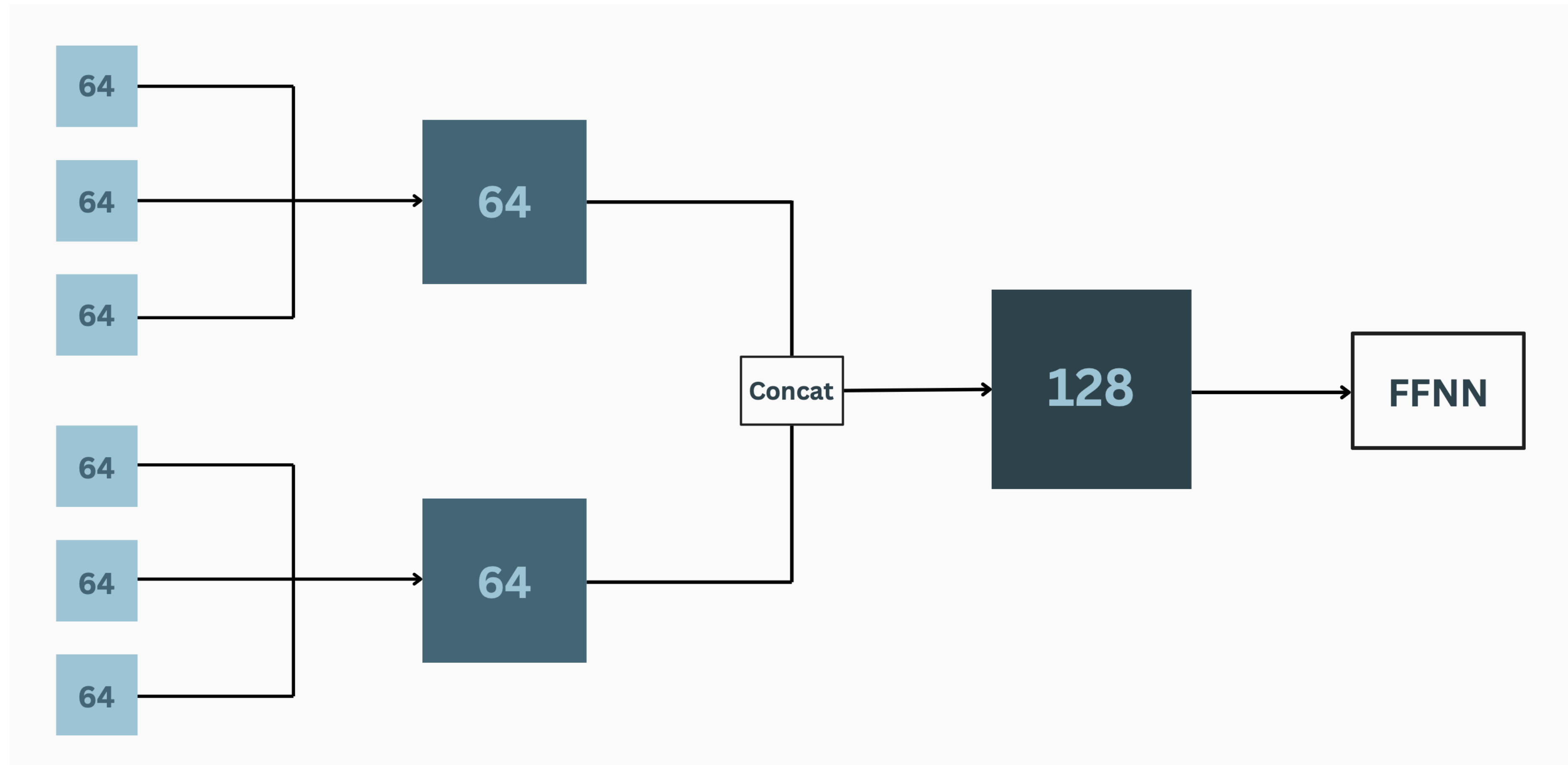
FFN

FFN takes a vector x (the hidden representation at a particular position in the sequence) and passes it through two learned linear transformations (represented by the matrices W_1 and W_2 and bias vectors b_1 and b_2) with ReLU

$$\mathbf{FFN}(x, W_1, W_2, b_1, b_2) = \max(0, xW_1 + b_1)W_2 + b_2$$



Transformer



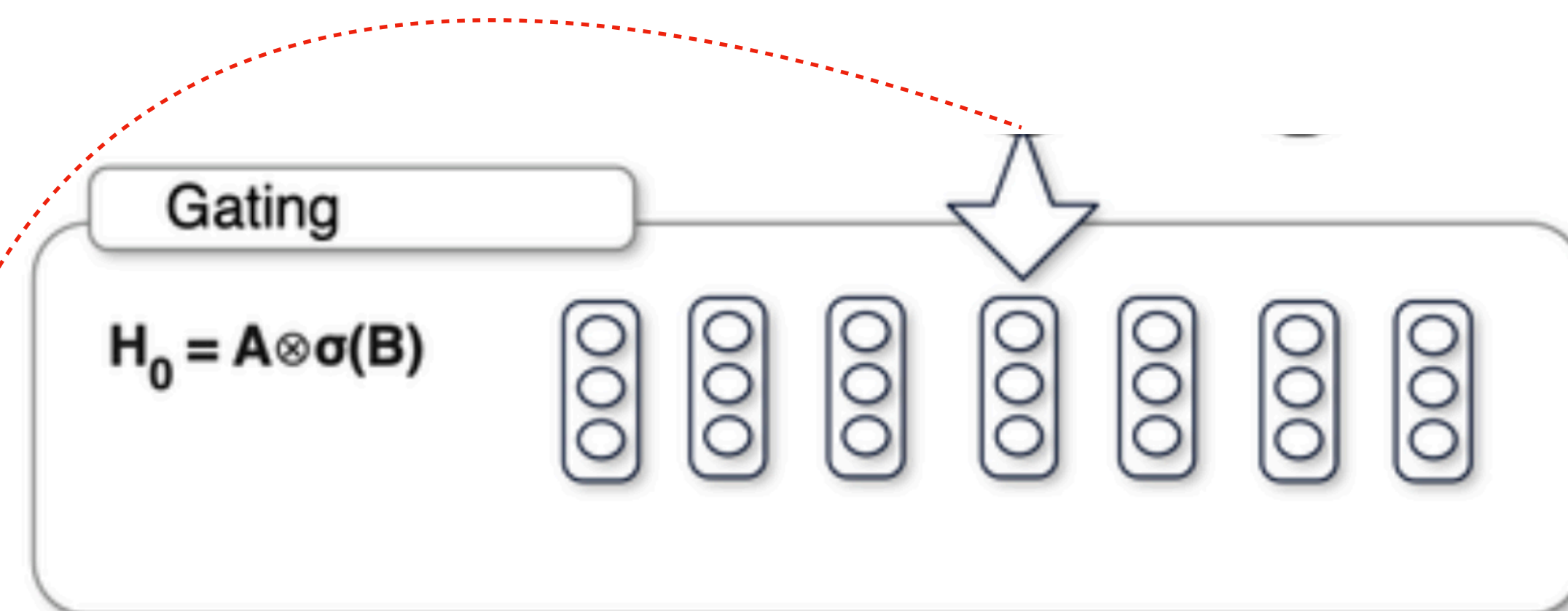
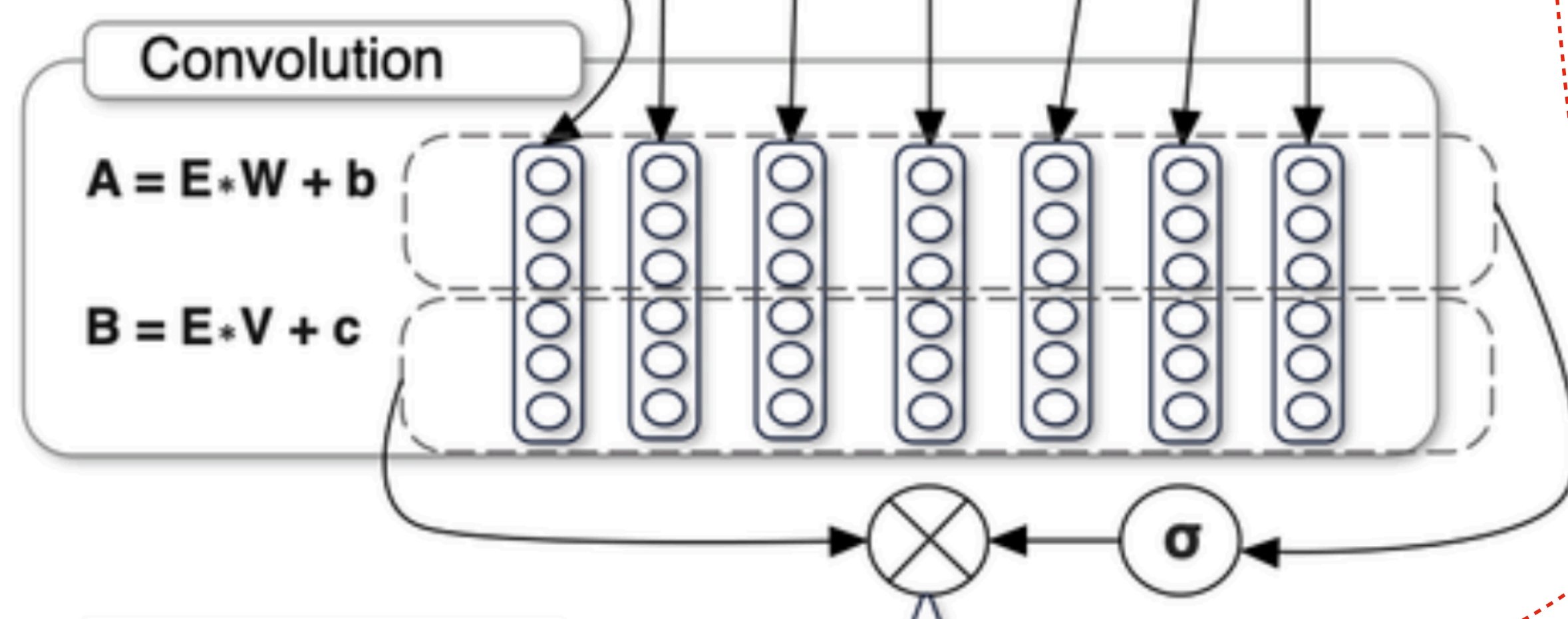
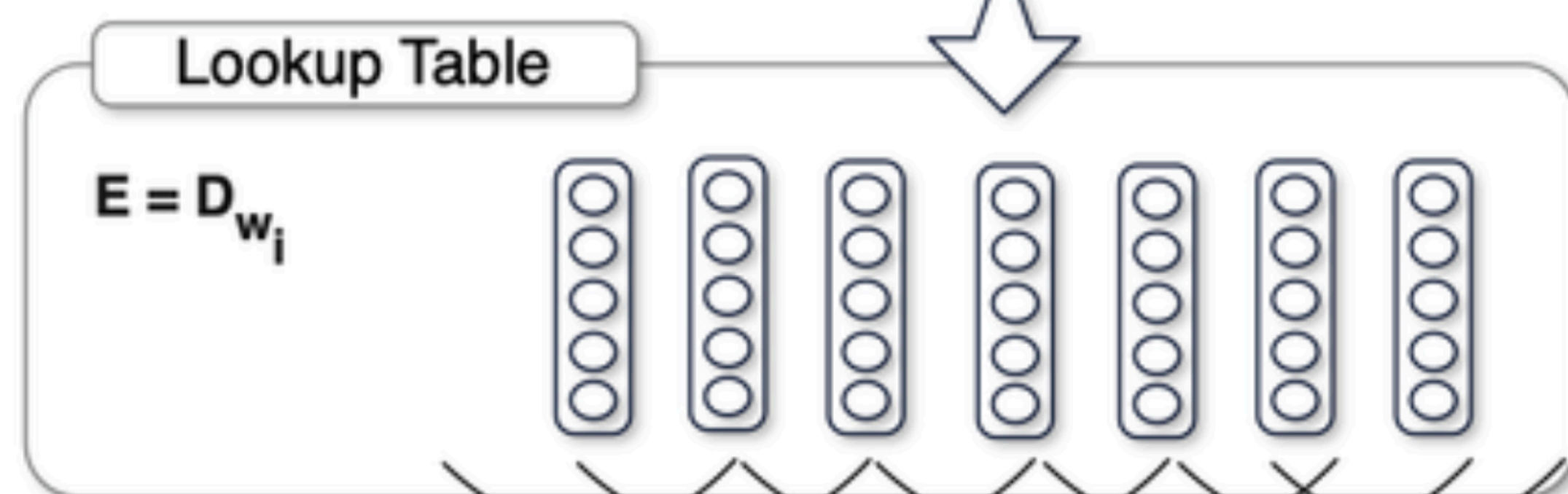
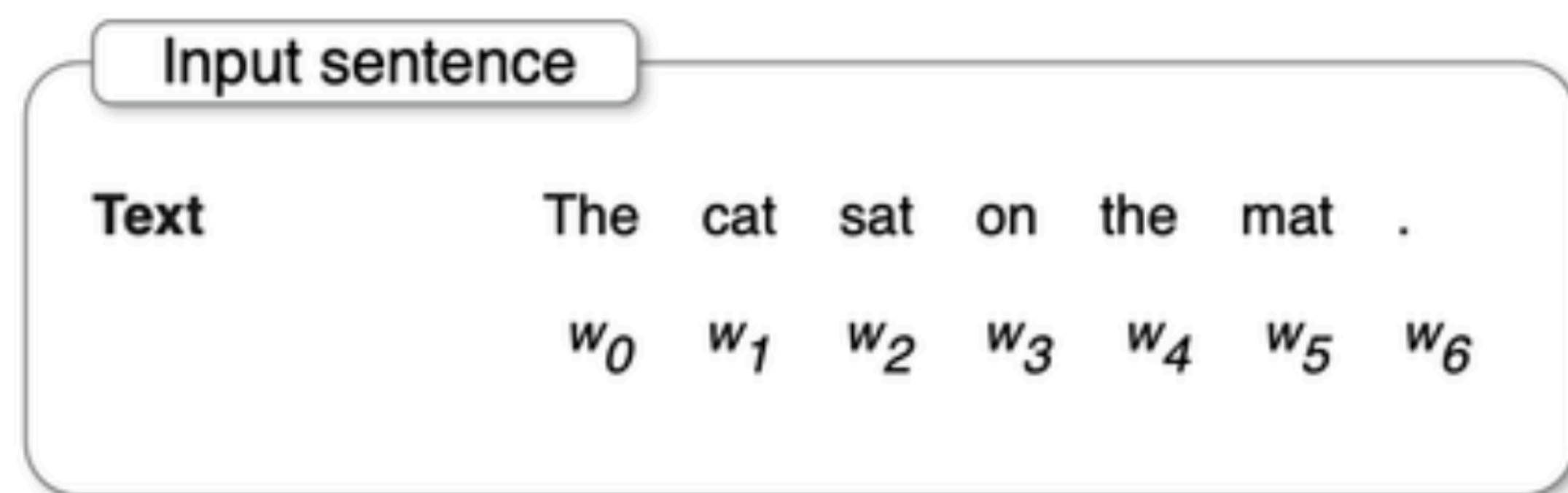
Gated Linear Units

Gating Principle

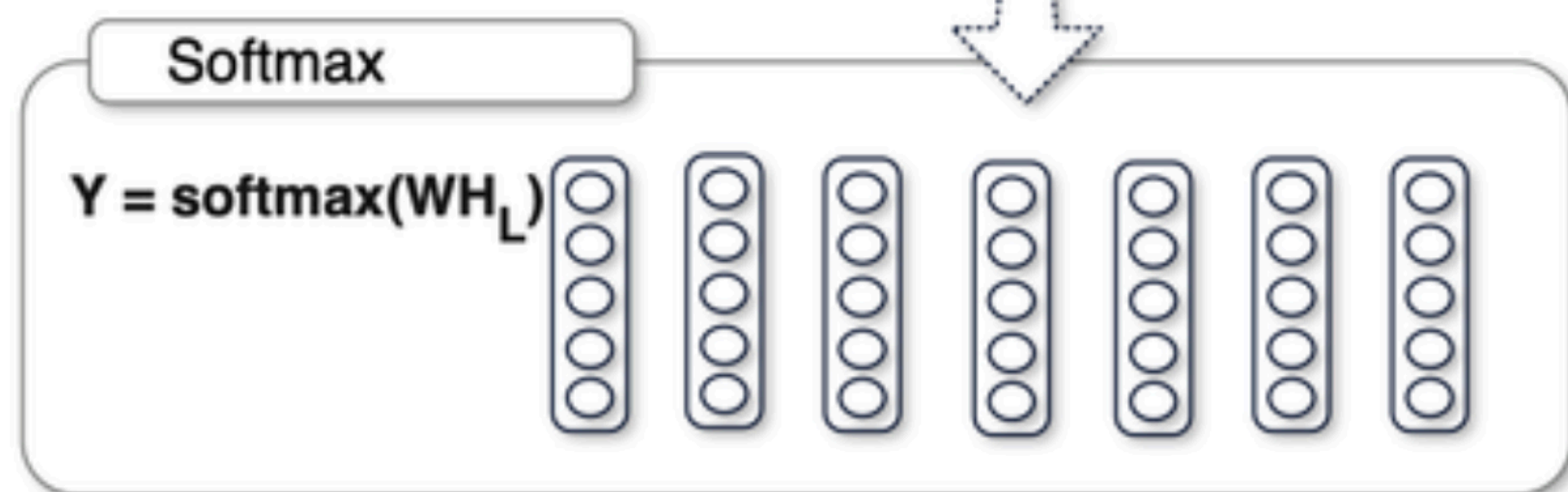
- Previously explored in GRUs and LSTMs.
- Motivation - create a learnable, input-dependent filter

$$\text{FFN}_{\text{GLU}}(x, W, V, W_2) = (\sigma(xW) \otimes xV)W_2$$

- **Gating:** x is projected by W , through a sigmoid function, producing a vector of values in the range $(0,1)$
- Simultaneously, x is projected by V . The element-wise product ($\otimes$) of the gate and the second projection results in a filtered representation, which is then projected by W_2 .
- **Bilinear Variant:** Sigmoid function is removed



Stack L - 1 Convolution+Gating Blocks



Why do they work?

“We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.” - Noam Shazeer

Hypothesis:

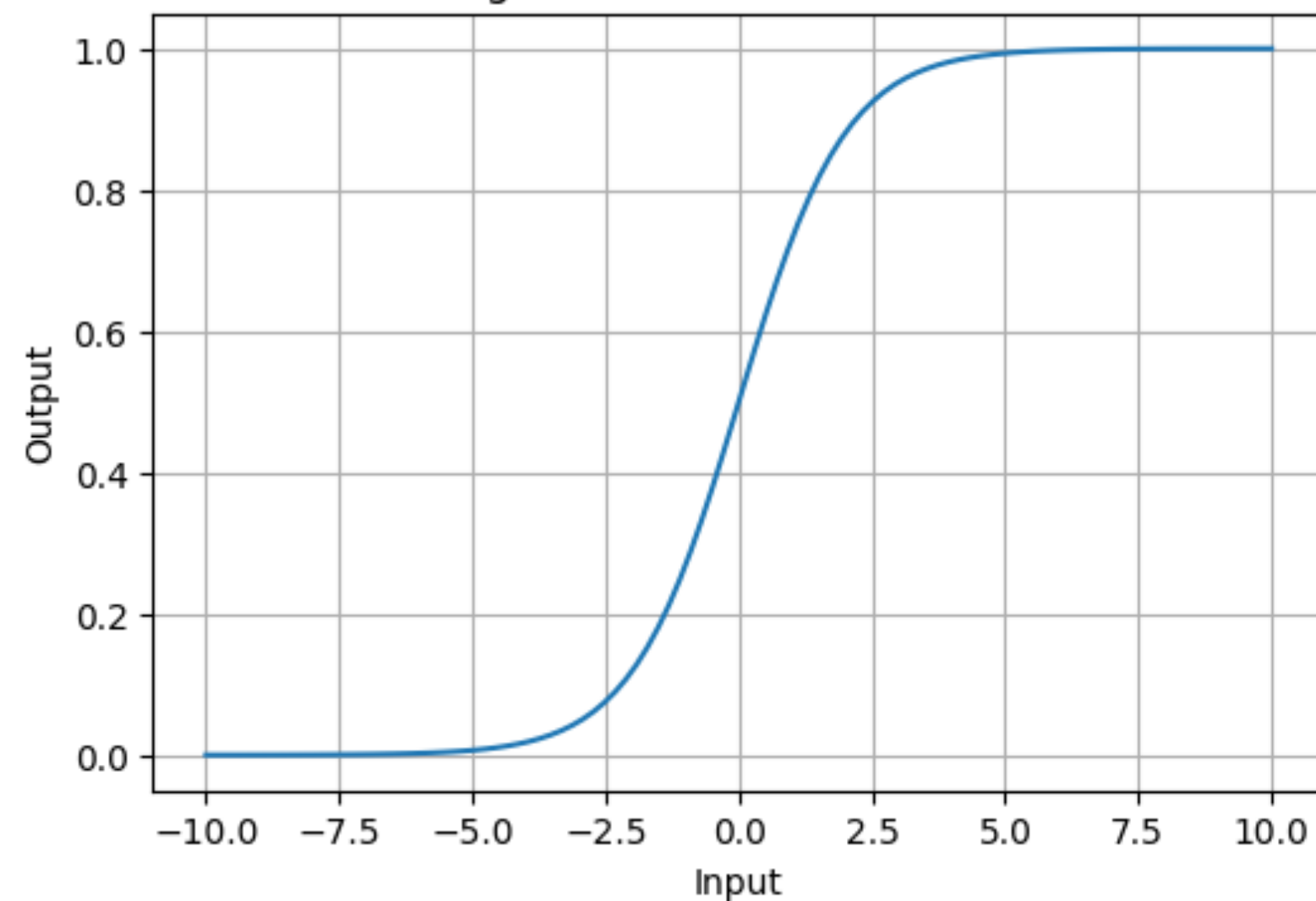
- Richer Representations: Input-dependent gating enables flexible, token-specific transformations
- Dynamic Feature Selection: Gates amplify relevant features and suppress irrelevant ones

GLU Variants

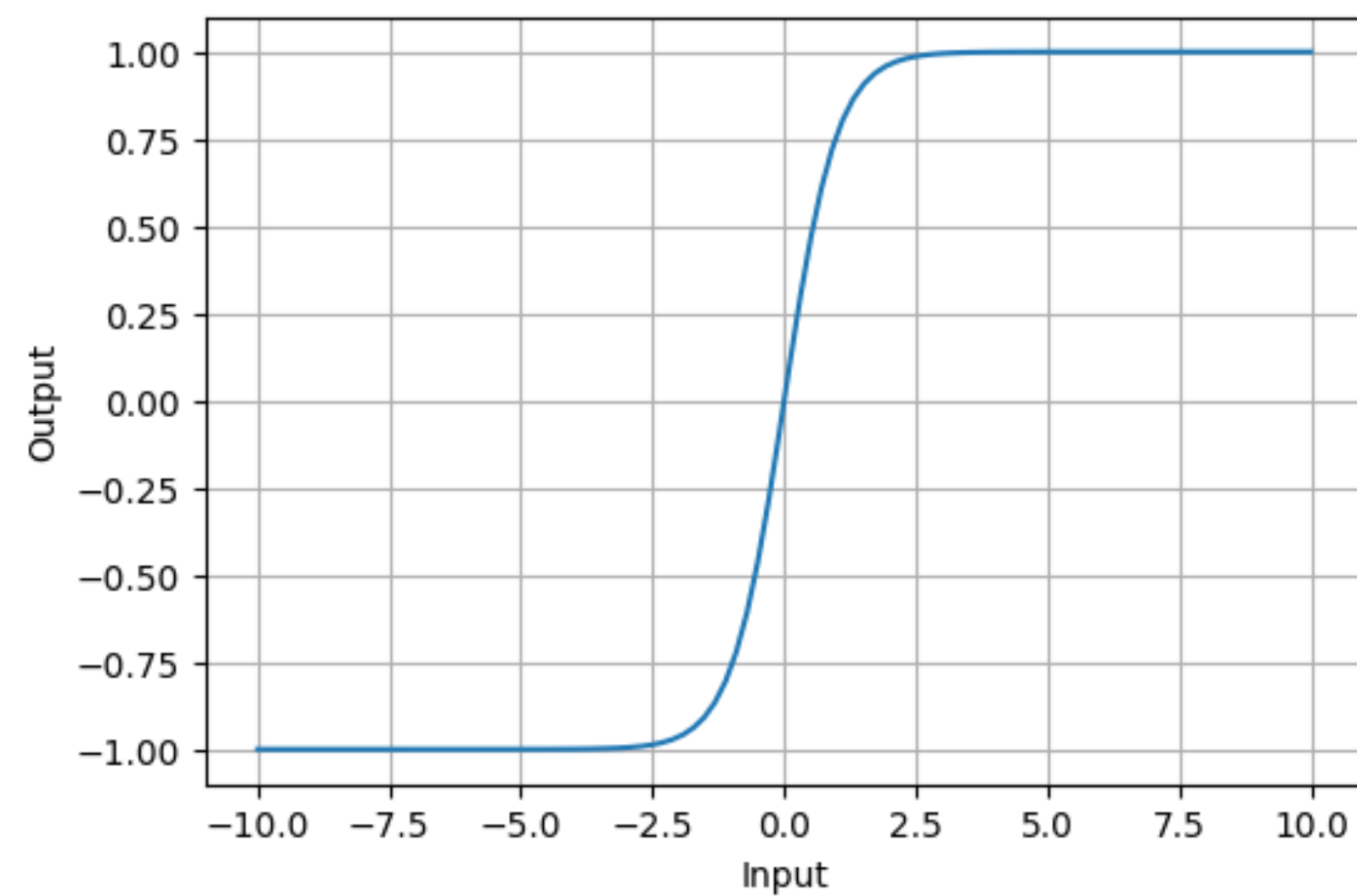
- GLU $\text{GLU}(x, W, V, b, c) = \sigma(xW + b) \otimes (xV + c)$
- Bilinear $\text{Bilinear}(x, W, V, b, c) = (xW + b) \otimes (xV + c)$
- ReGLU $\text{ReGLU}(x, W, V, b, c) = \max(0, xW + b) \otimes (xV + c)$
- GeGLU $\text{GeGLU}(x, W, V, b, c) = \text{GELU}(xW + b) \otimes (xV + c)$
- ★ SwiGLU $\text{SwishGLU}(x, W, V, b, c) = \text{Swish}_{\beta}(xW + b) \otimes (xV + c)$

$$\text{Swish}_{\beta}(x) = x\sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}$$

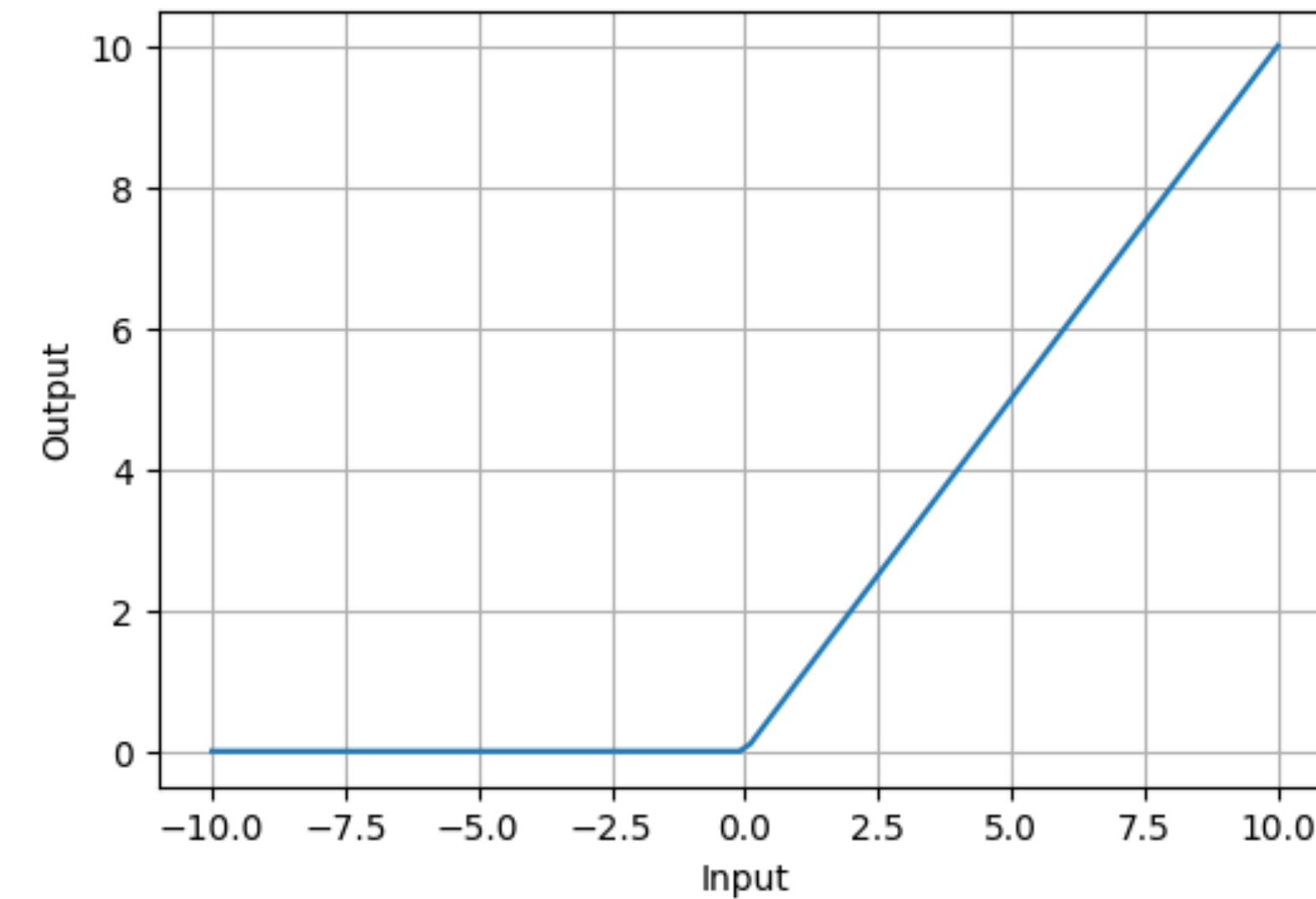
Sigmoid Activation Function



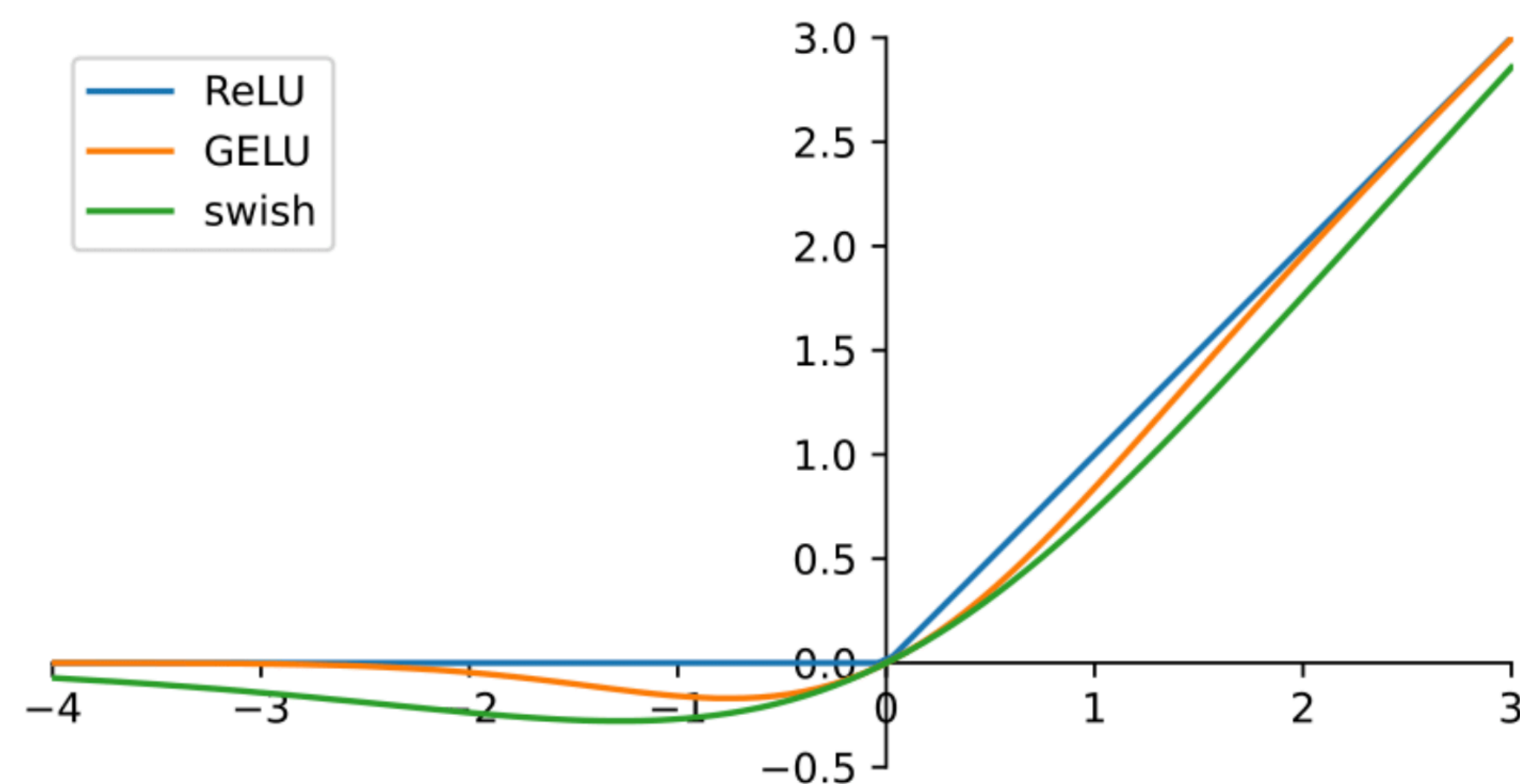
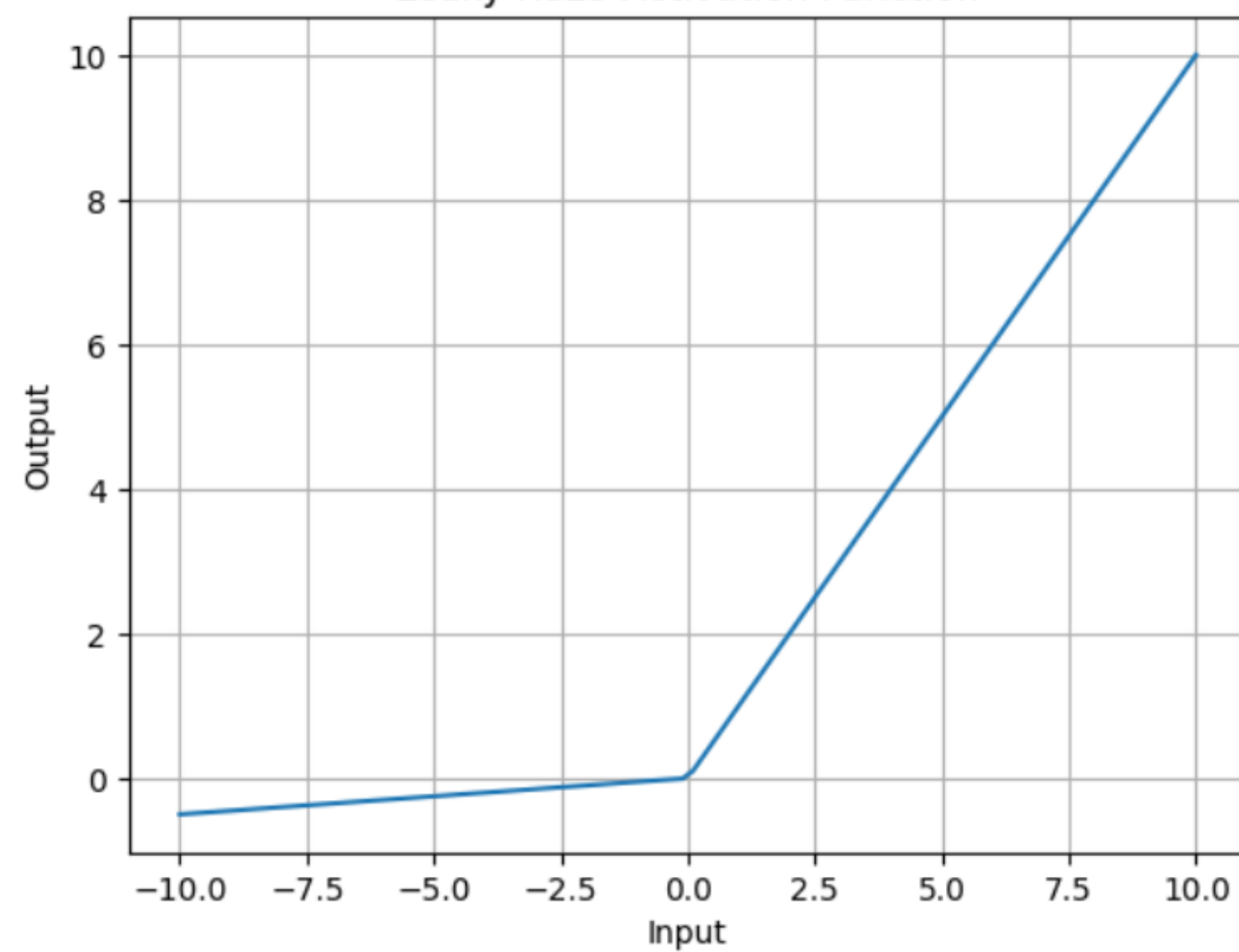
Tanh Activation Function



ReLU Activation Function



Leaky ReLU Activation Function



3 weight matrices

- All GLU variants use 3 weight matrices (W , V , W_2), against the standard 2
- To tackle this increase in parameters, we reduce the number of hidden units (d_{ff}) by a factor of $\frac{2}{3}$



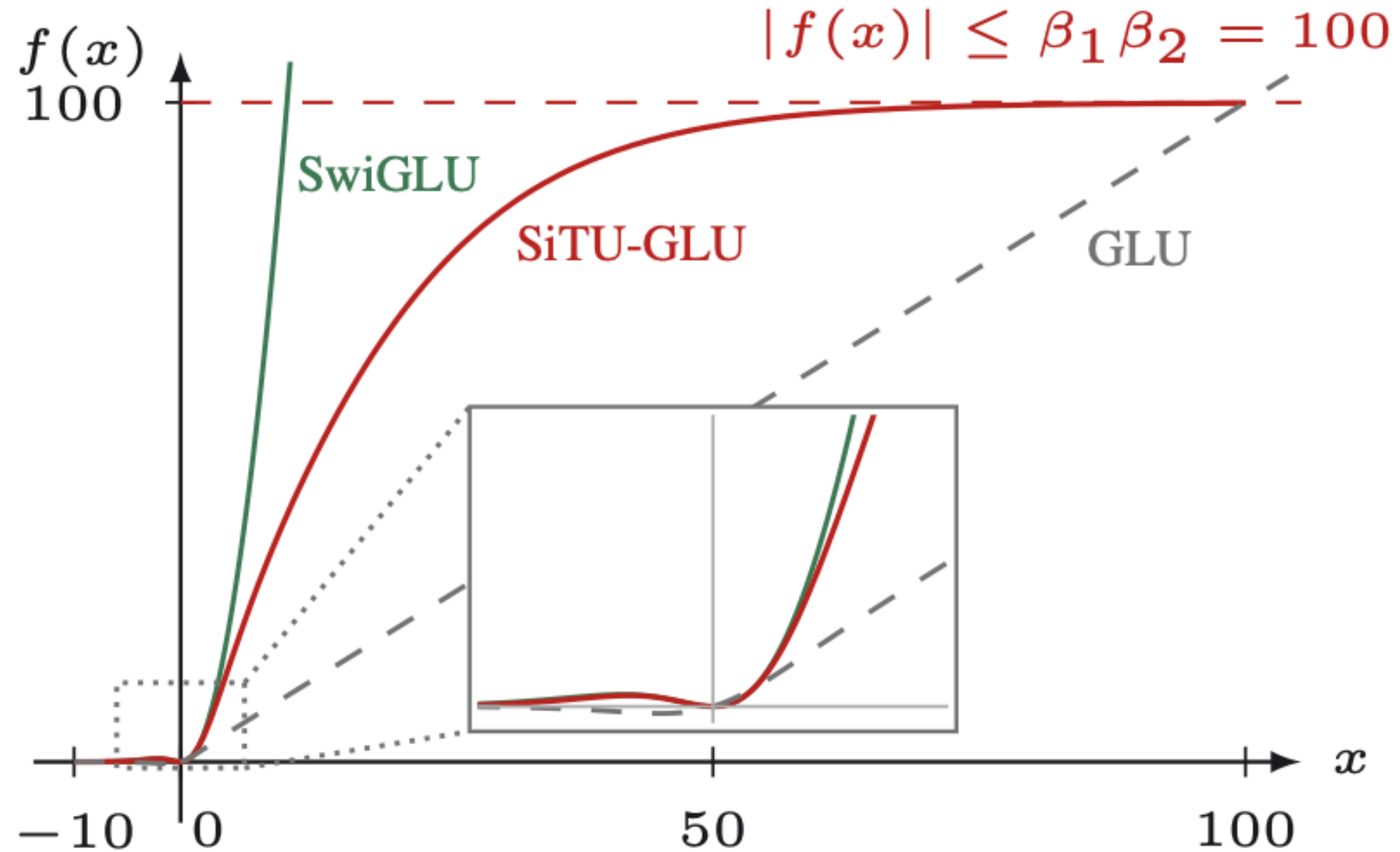
SiTU-GLU

Sigmoid Tanh Unit - GLU

- Kimi K3 proposed a new function! 🎉
- applies the smooth cap softcap(x, β) = $\beta \tanh(x/\beta)$ to the linear factor of the Swish gate and independently to the up branch

$$\text{SiTU - GLU}(x) = \left[\beta_1 \tanh \left(\frac{W_g x}{\beta_1} \right) \odot \sigma(W_g x) \right] \odot \left[\beta_2 \tanh \left(\frac{W_u x}{\beta_2} \right) \right]$$

SiTU-GLU



Activation Spikes

- Recent research shows that GLU gating can cause extreme "activation spikes" especially in early and late layers
- They are harmless in full-precision training
- These outliers hinder post-training quantization, as standard schemes can't capture both typical values and spikes



FFNN

Issues

- Static, Dense design that is not content aware
- Uniform Computational Load
 - Applies the same massive computation to every token, this is highly inefficient
- Static Knowledge Access
 - Activates the entire FFN parameter set for every token, regardless of the knowledge needed



Scaling through Sparsity

- Scaling Laws: as the number of parameters, the size of the training dataset, and the amount of compute increase, model performance predictably improves. But, this is not feasible for everyone.
- MoE introduces the principle of sparse, conditional computation to the Transformer, allowing models to grow to enormous parameter counts while keeping the computational cost for any single input manageable.



What are MoEs

- Technique that uses multiple sub-models (“experts”)
- Scaling through Sparsity



MoE

Advantages

- Sparse models -> Much larger parameters -> More information
 - You can increase the number experts without affecting FLOPs
- Inherently parallel (Expert Parallelism)
- Sparse models are good to train with data parallelism
 - Training on machines which can host all parameters



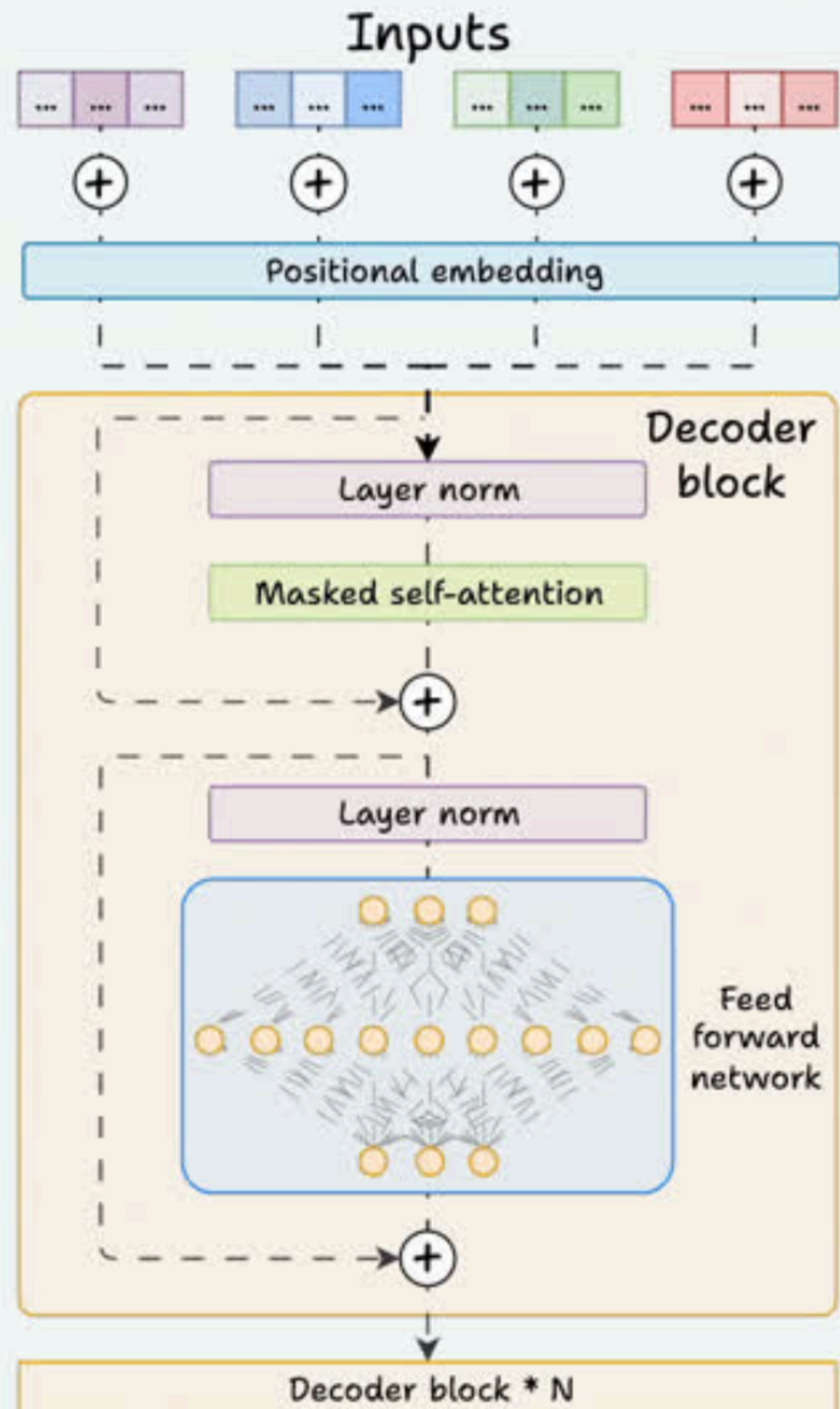
MoE

Why was it not popular

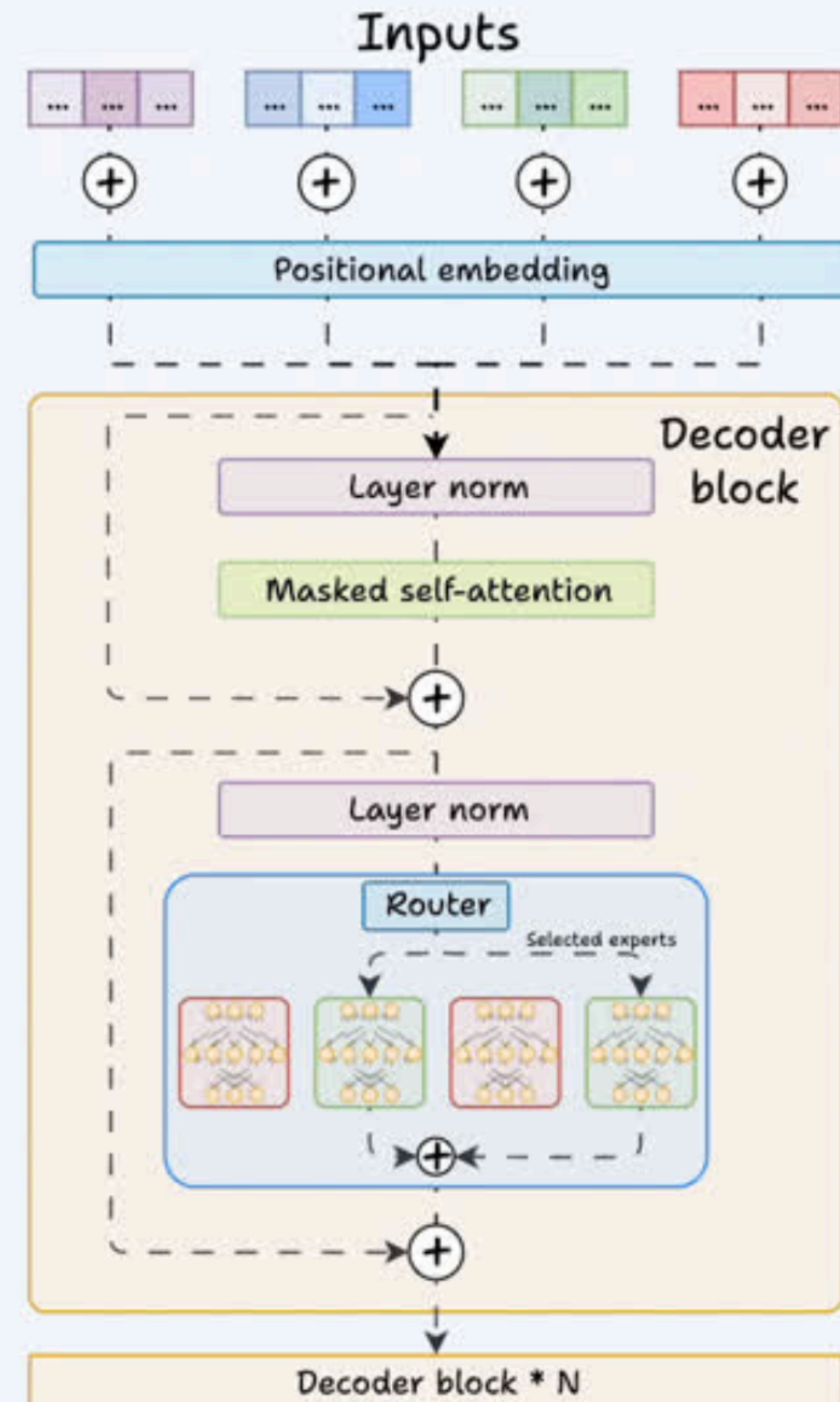
- Complex Architecture
- Training objectives are somewhat heuristic
- Unstable to train



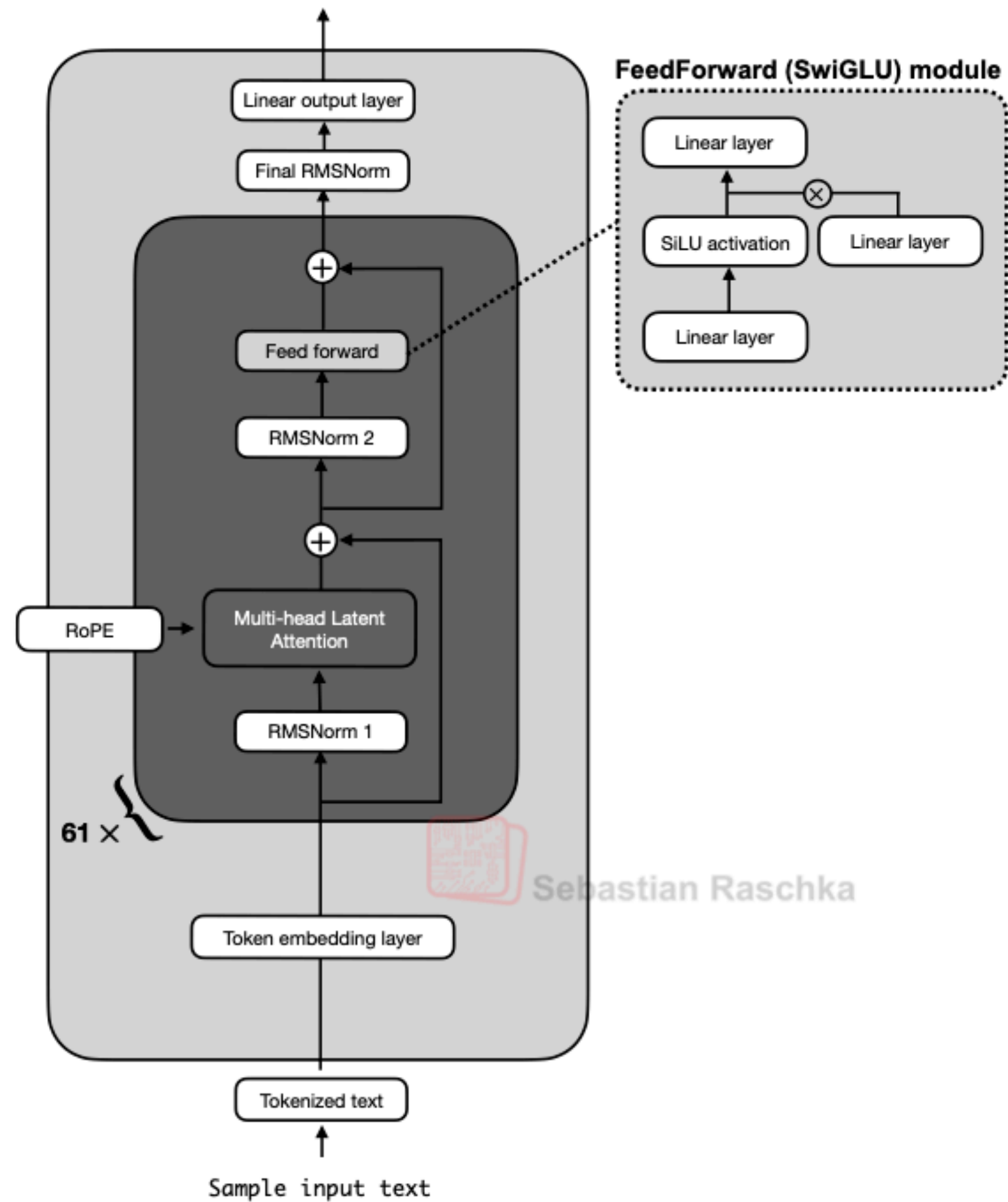
Transformer



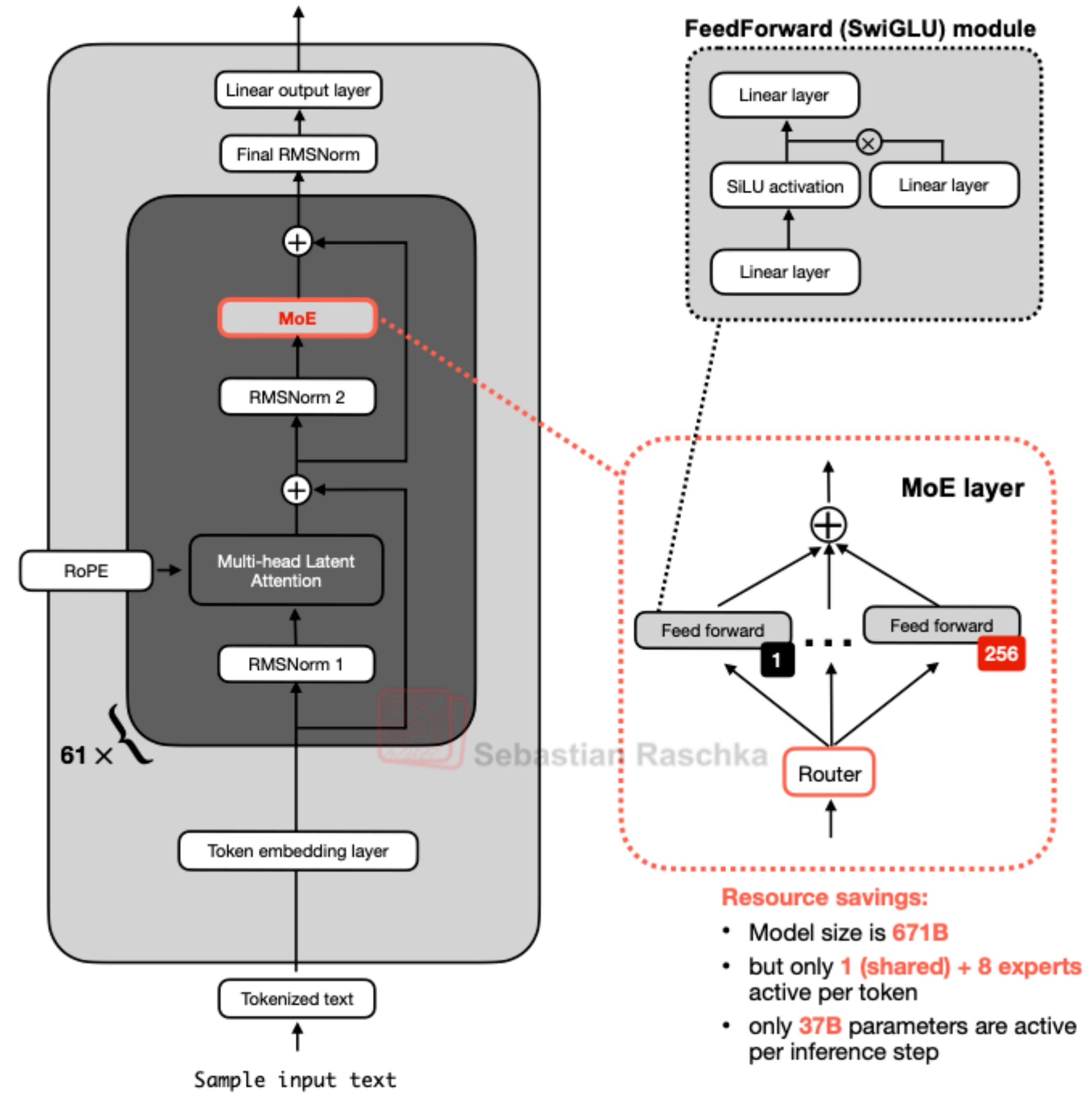
Mixture of Experts



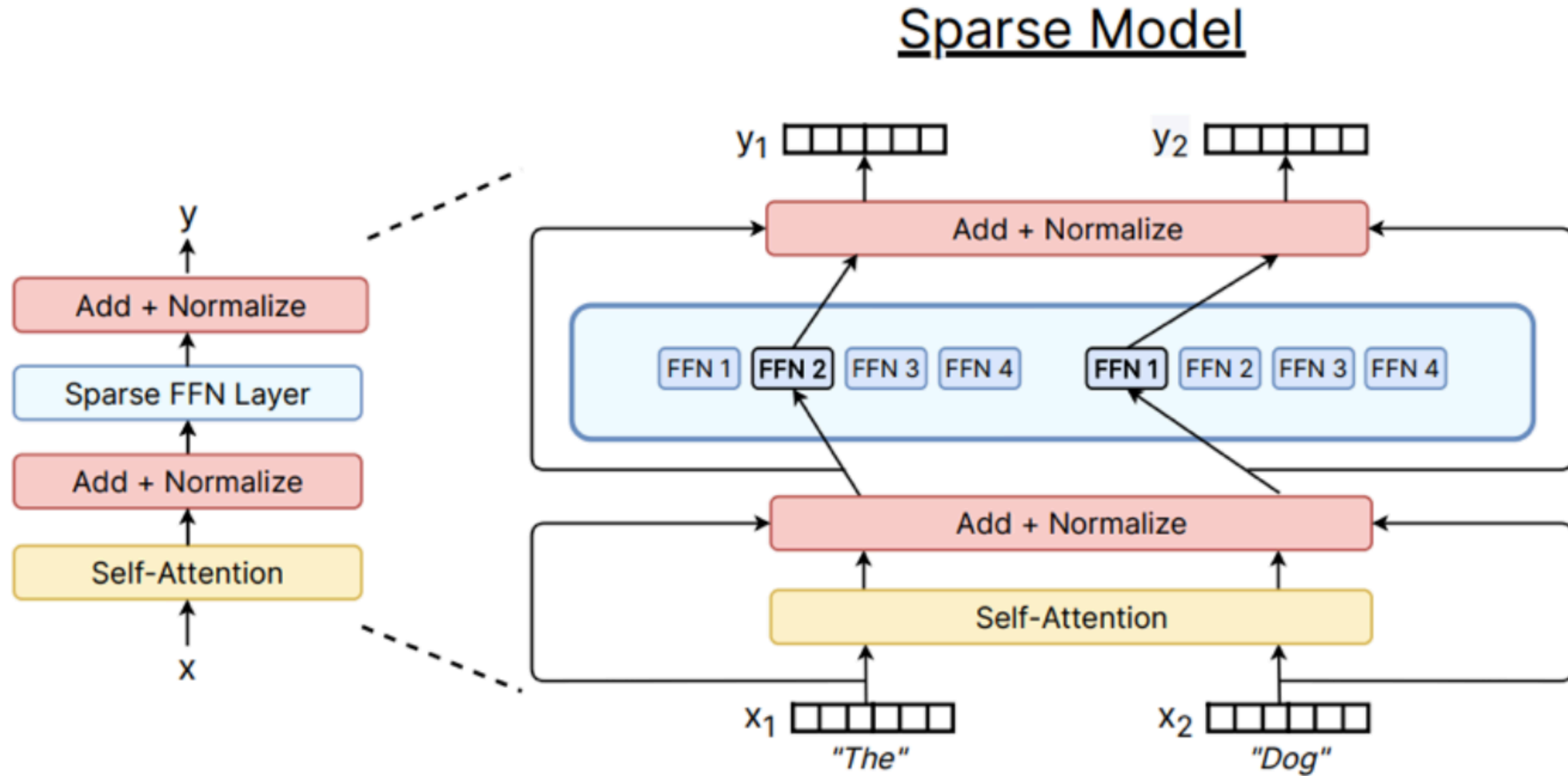
Architecture without MoE (“dense”)



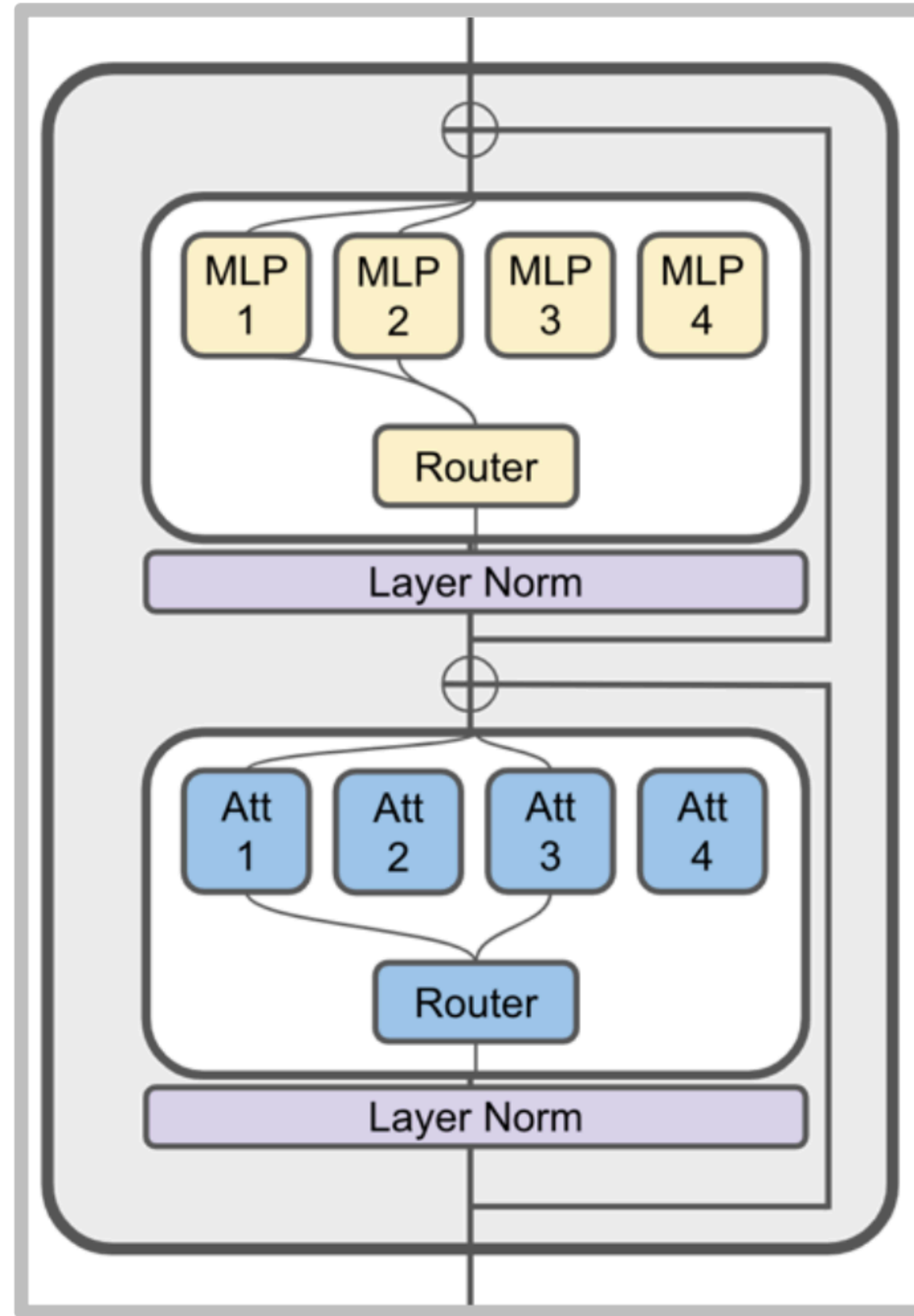
DeepSeek V3/R1 with MoE (“sparse”)



Typically replace MLP with MoE Layer



Less Common Use MoE for Attention Heads



MoE

Components

- Experts
- Router/Gate
- Load Balancer



MoE

What can vary

- Expert sizes
- Routing Algorithms
- Training Objectives



Experts

Experts

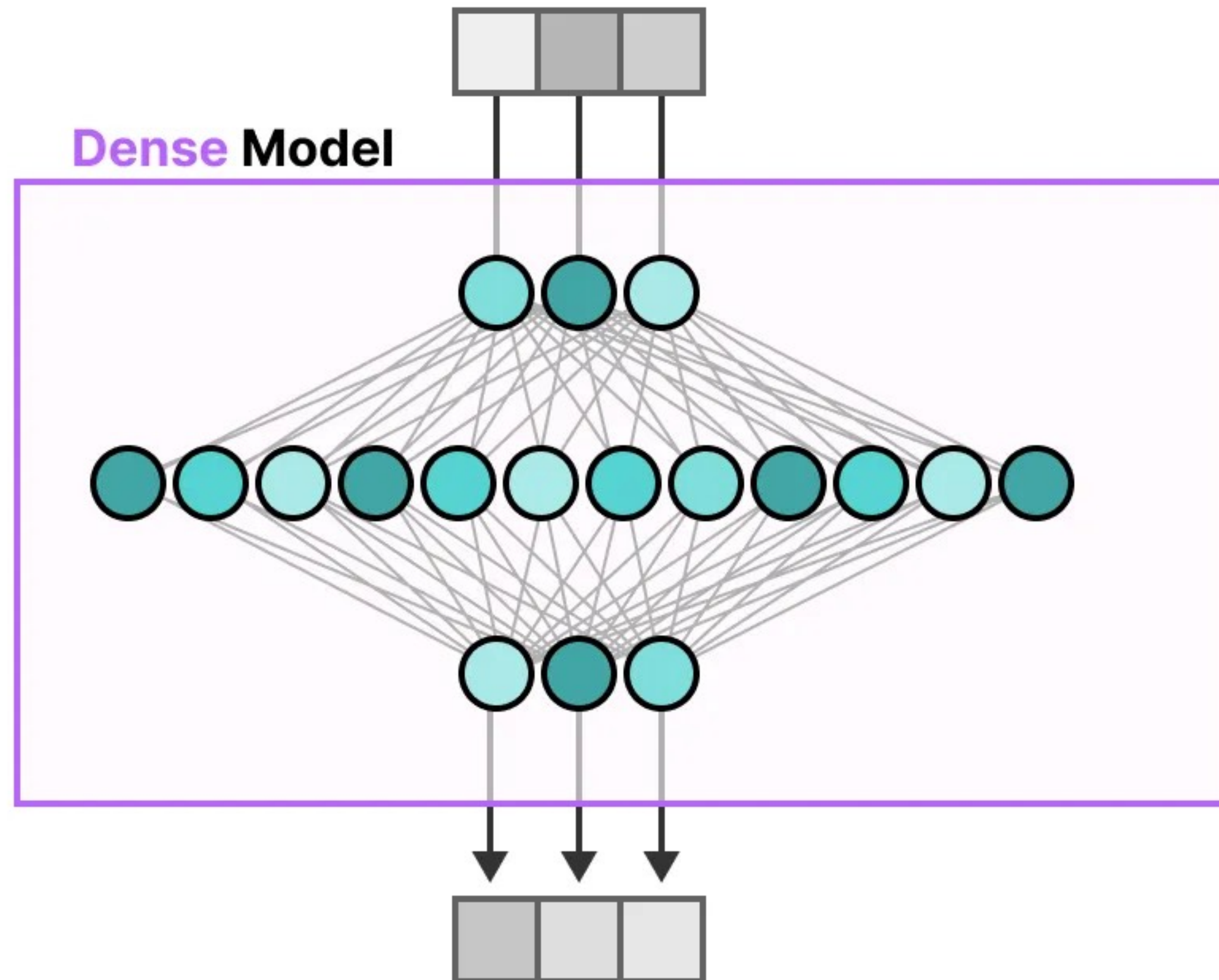
- What do the experts actually learn?
- How do we ensure complementary and non-redundant knowledge?
- Is it really “divine benevolence”?
- How do we interpret these complex models?

Experts

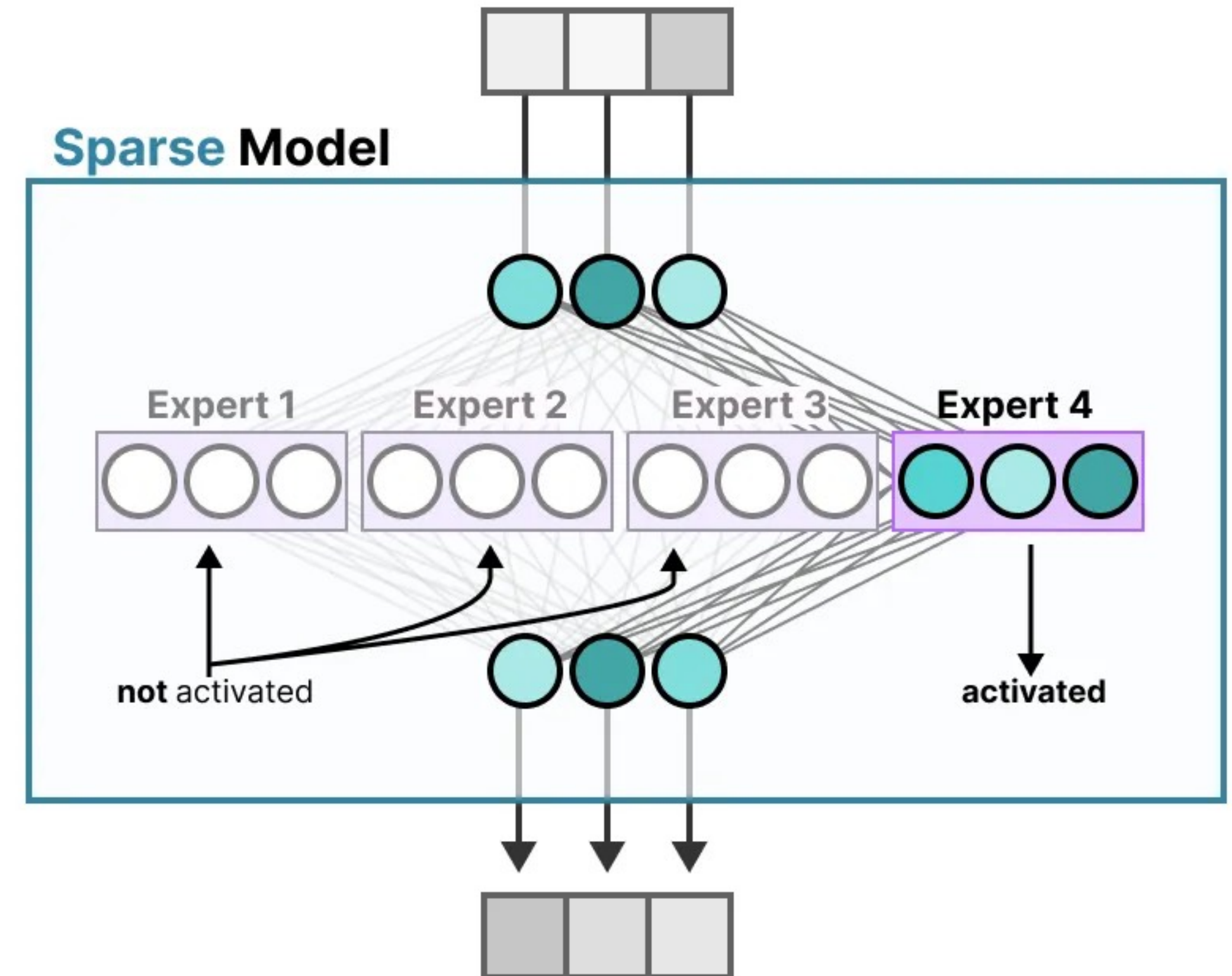
- Sort of a “misnomer”
- At least one expert is activated in the process
- Experts are typically FFNNs
- Specialize on different things during training



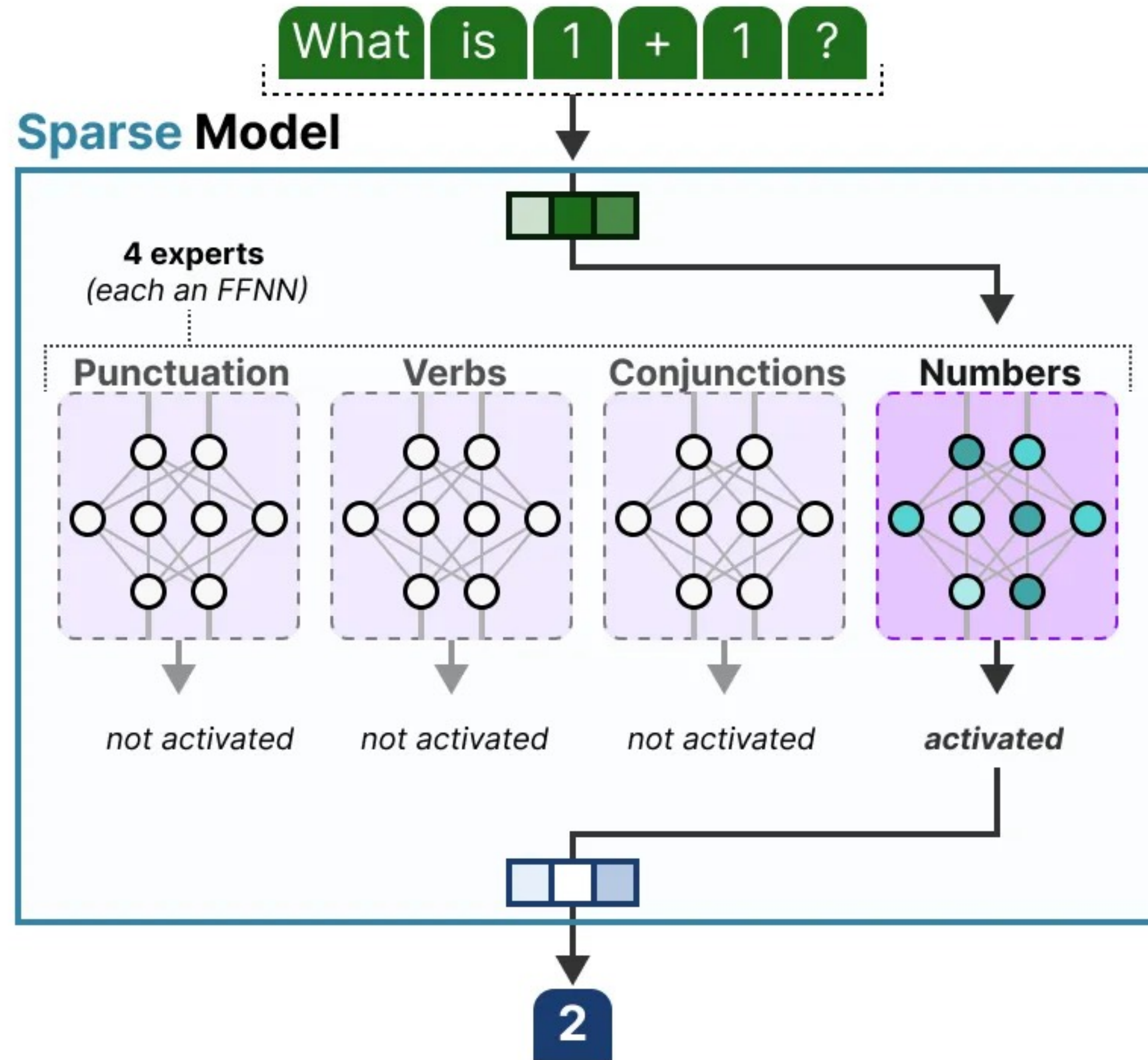
Dense Model



Sparse Model

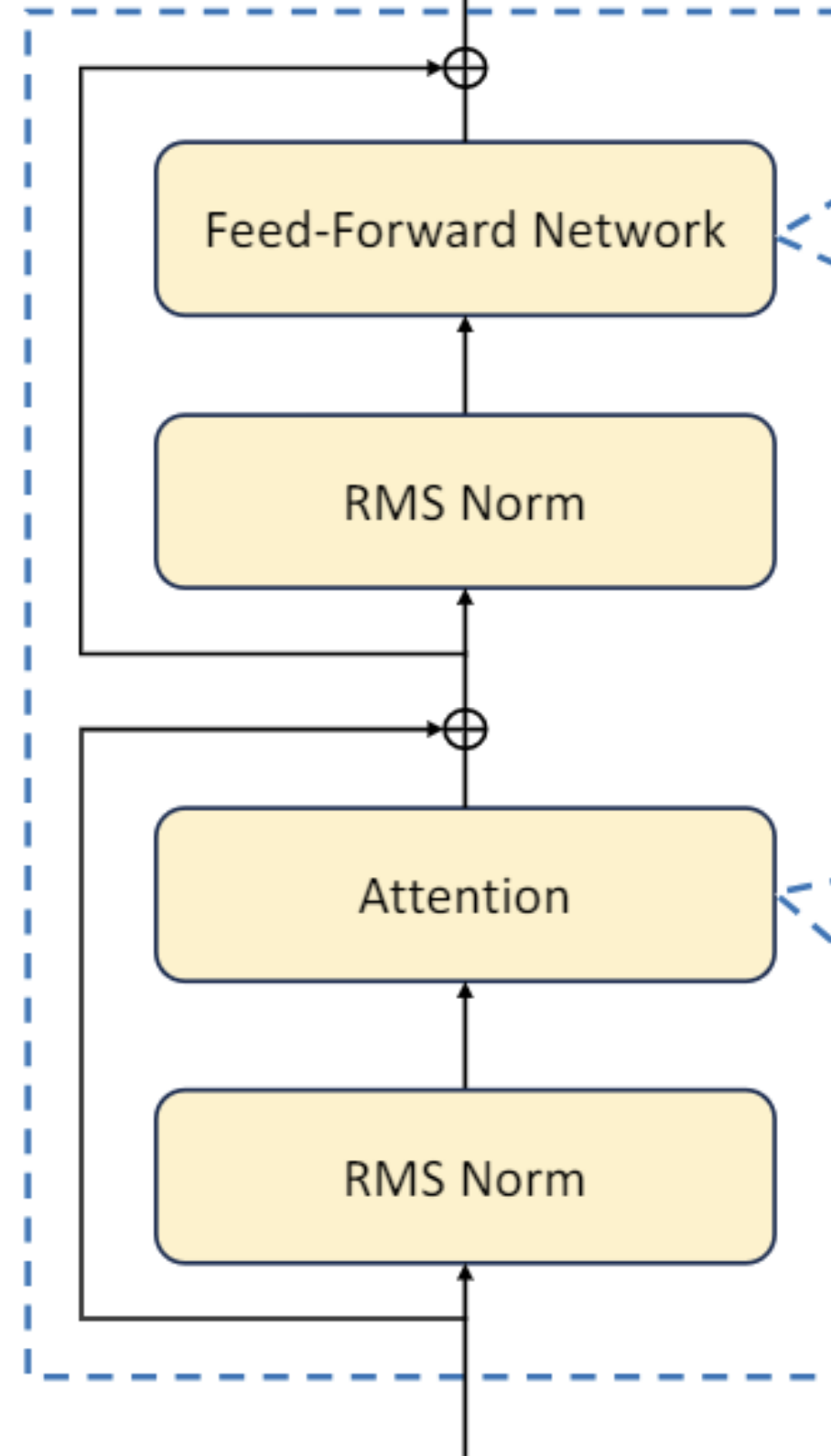


Expert specialization	Expert position	Routed tokens
Sentinel tokens	Layer 1	been <extra_id_4><extra_id_7>floral to <extra_id_10><extra_id_12><extra_id_15> <extra_id_17><extra_id_18><extra_id_19>...
	Layer 4	<extra_id_0><extra_id_1><extra_id_2> <extra_id_4><extra_id_6><extra_id_7> <extra_id_12><extra_id_13><extra_id_14>...
	Layer 6	<extra_id_0><extra_id_4><extra_id_5> <extra_id_6><extra_id_7><extra_id_14> <extra_id_16><extra_id_17><extra_id_18>...
Punctuation	Layer 2	, , , , , , , , - , , , , ,) .)
	Layer 6	, , , , , : : : , & , & & ? & - , , ? , , , . <extra_id_27>
Conjunctions and articles	Layer 3	The the the the the the the the the The the the the the the The the the the
	Layer 6	a and and and and and and and or and a and . the the if ? a designed does been is not
Verbs	Layer 1	died falling identified fell closed left posted lost felt left said read miss place struggling falling signed died falling designed based disagree submitted develop
Visual descriptions <i>color, spatial position</i>	Layer 0	her over her know dark upper dark outer center upper blue inner yellow raw mama bright bright over open your dark blue
Proper names	Layer 1	A Mart Gr Mart Kent Med Cor Tri Ca Mart R Mart Lorraine Colin Ken Sam Ken Gr Angel A Dou Now Ga GT Q Ga C Ko C Ko Ga G
Counting and numbers <i>written and numerical forms</i>	Layer 1	after 37 19. 6. 27 I I Seven 25 4, 54 I two dead we Some 2012 who we few lower each

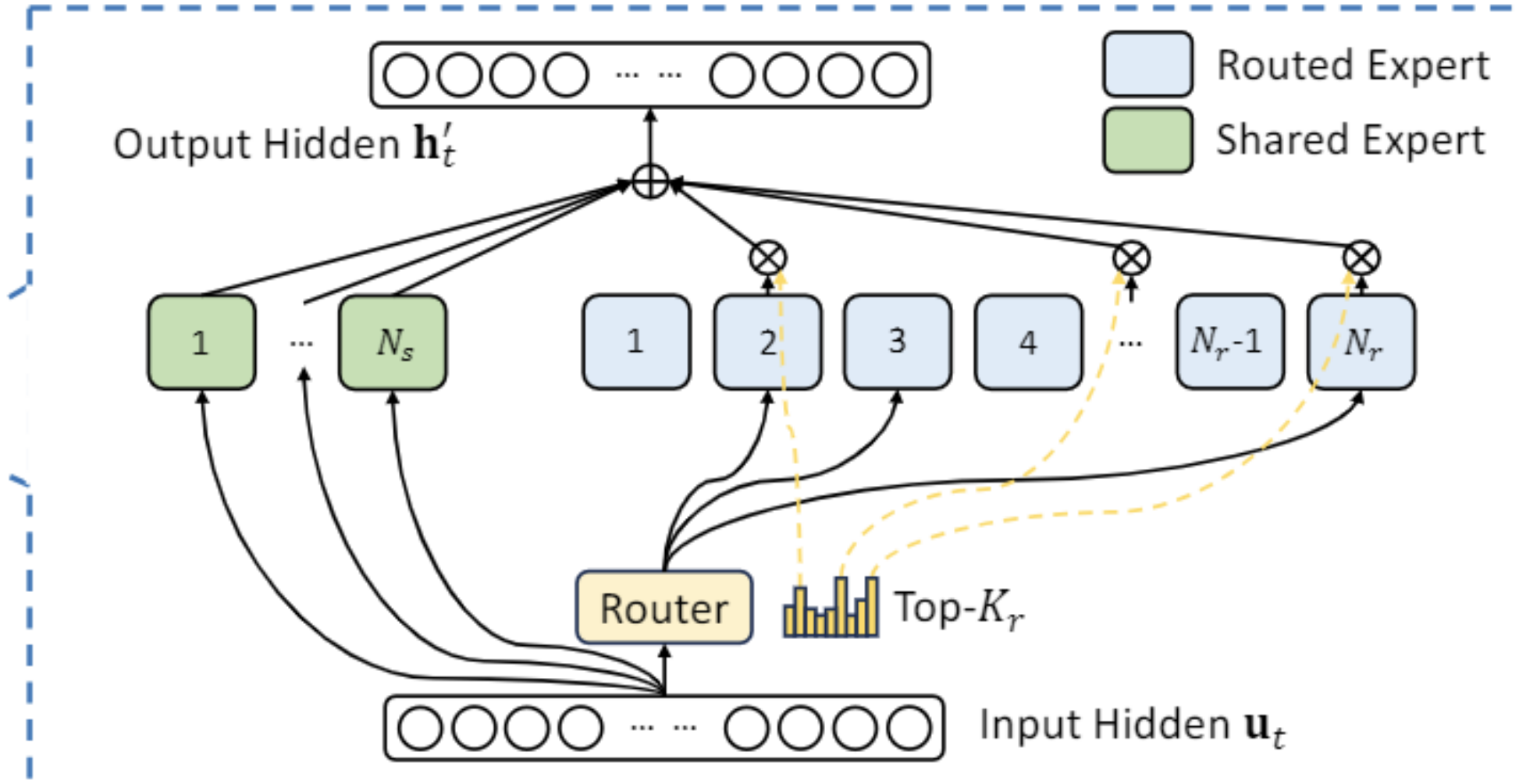


DeepSeek MoE

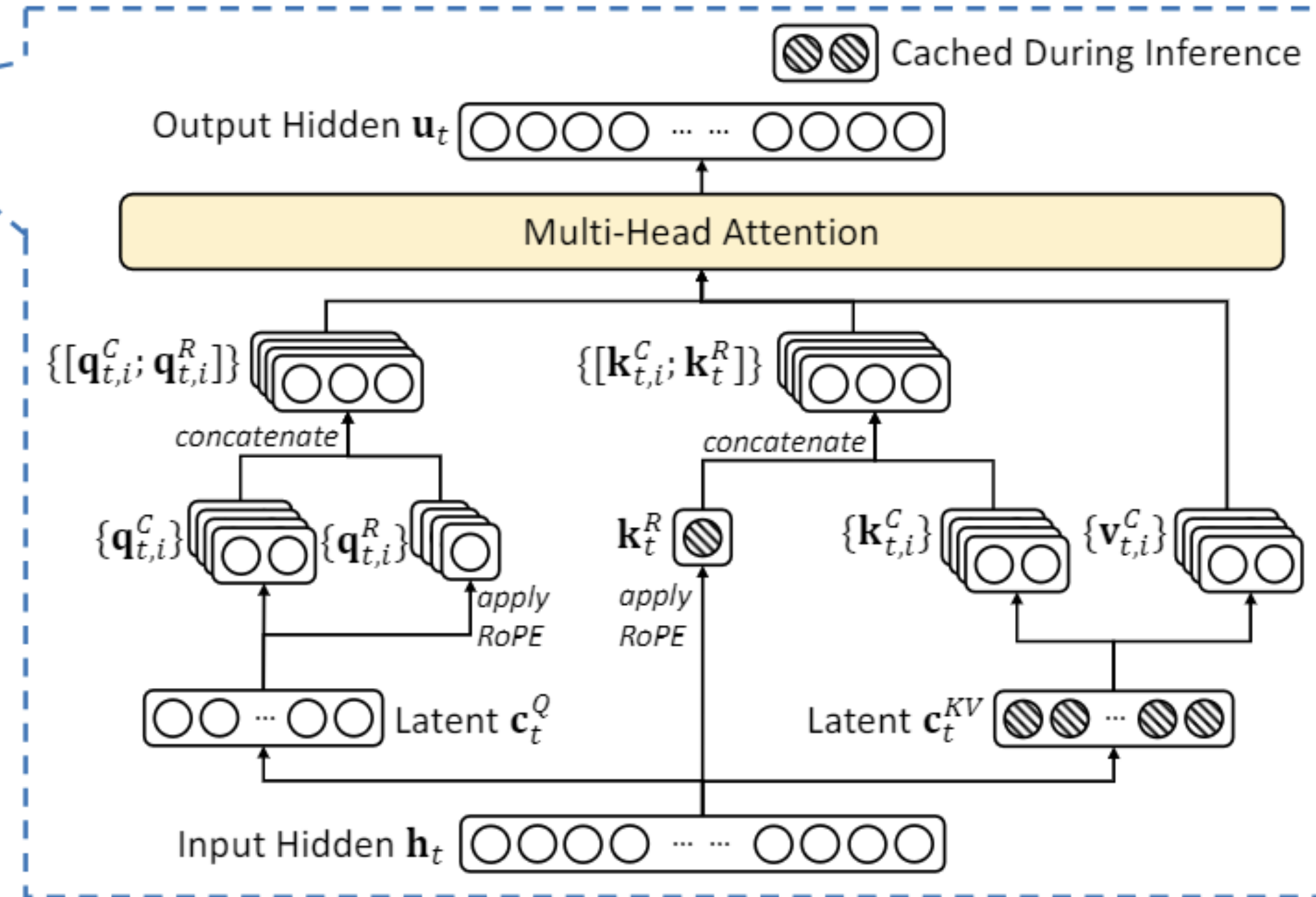
Transformer Block $\times L$



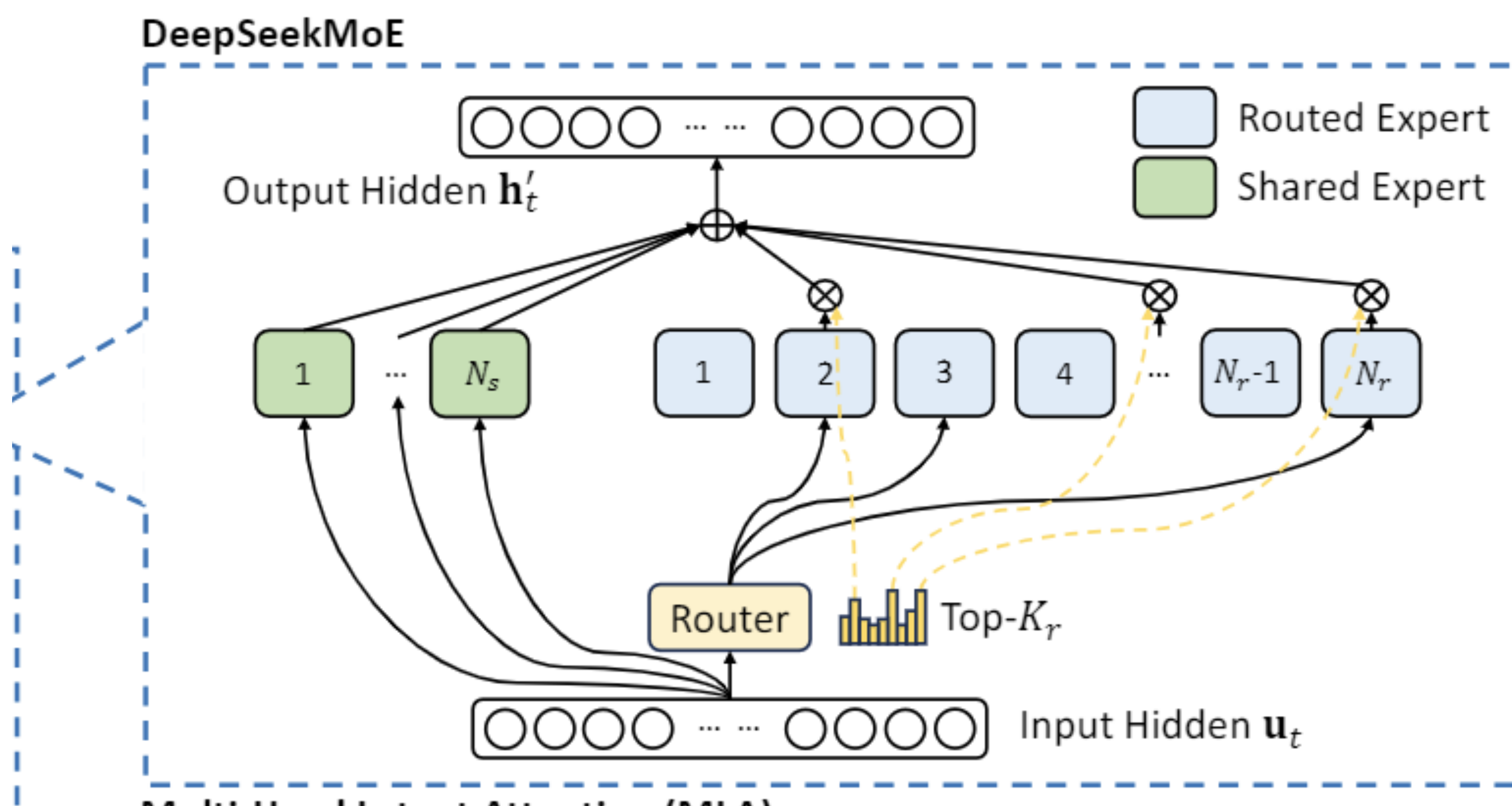
DeepSeekMoE



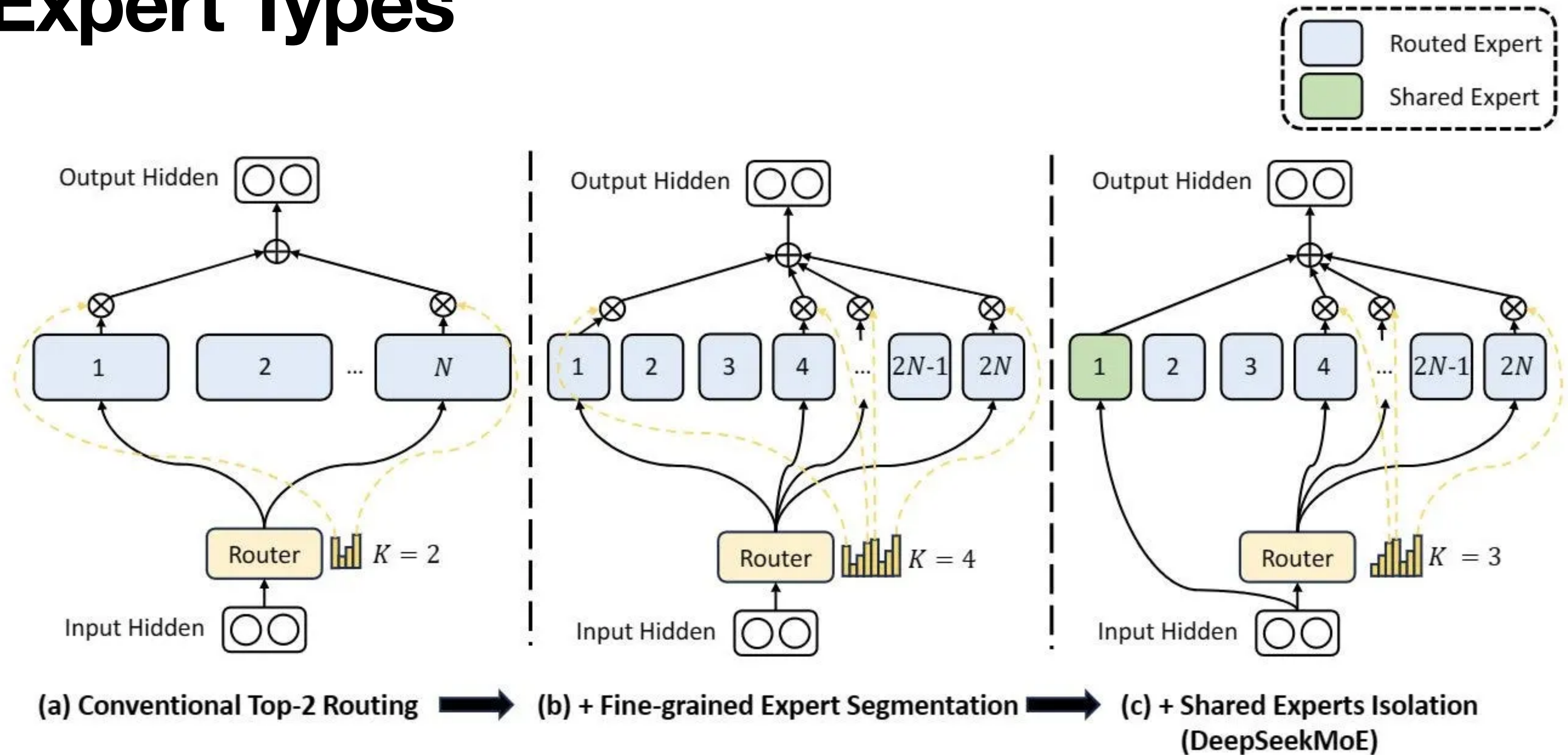
Multi-Head Latent Attention (MLA)



DeepSeek MoE



Expert Types



Smaller (size), larger (count) number of routed experts + a few shared experts that are always on.

Routing

Routing/Gating

- Most critical component of MoE architecture
- Used to choose the expert
- Also a FFNN
- Outputs probabilities which are used to select the best matching expert

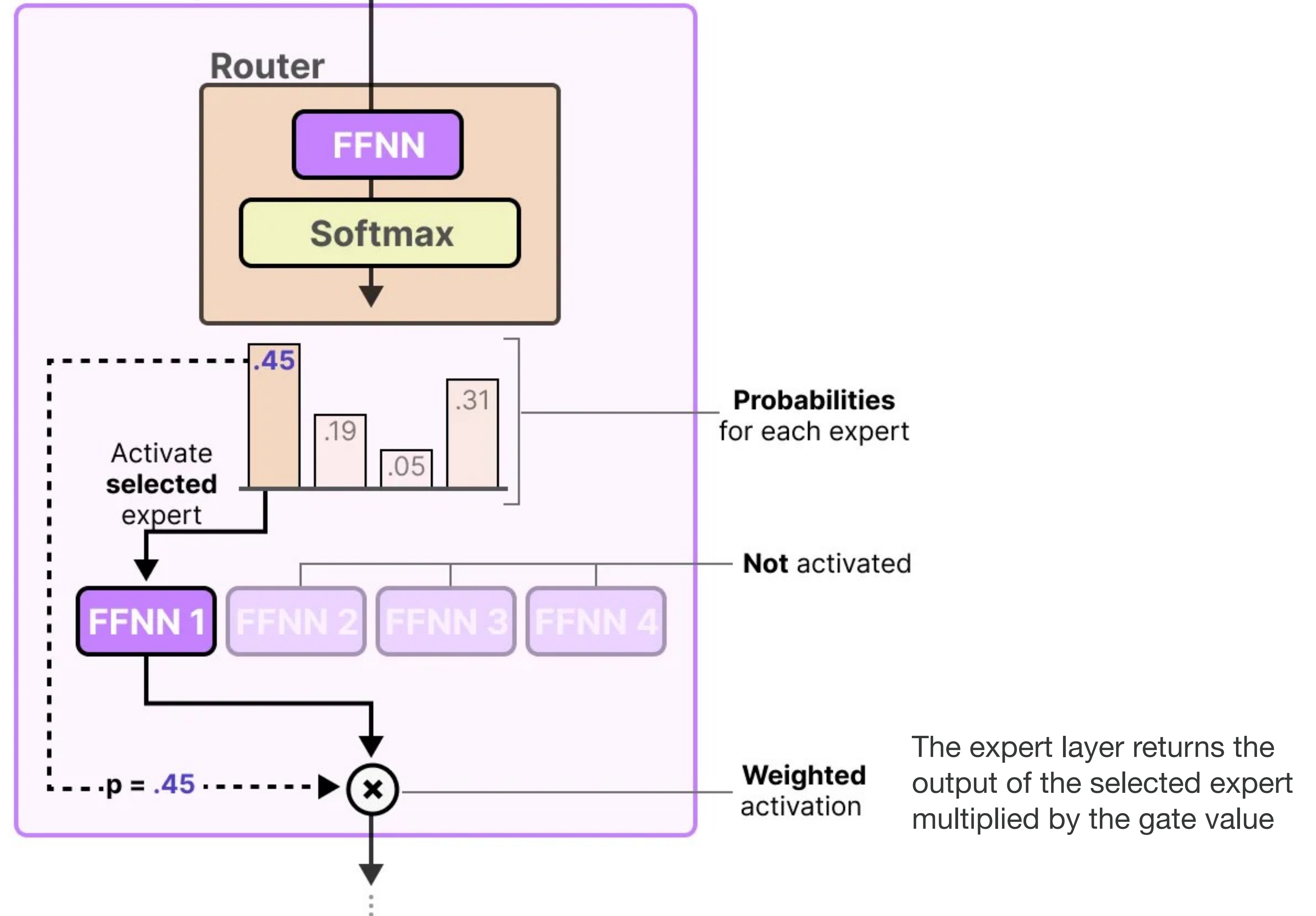


Routing Algorithms

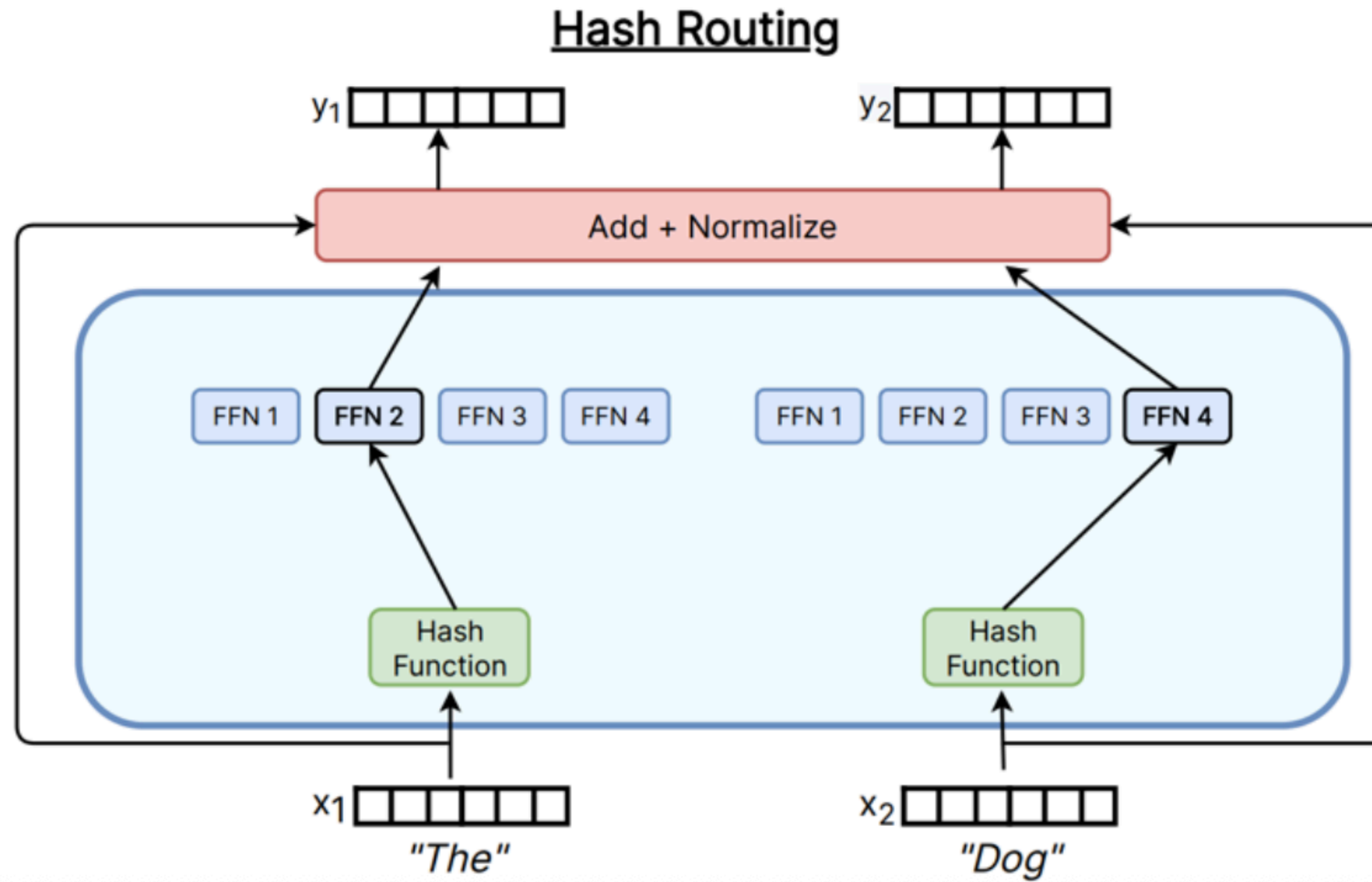
- Hash Routing (Mostly Academic Use)
- Top K Routing
 - Token Choice
 - Expert Choice
- Reinforcement Learning



MoE Layer



Hash Routing



Top k Routing

		Tokens		
		T1	T2	T3
Experts	E1	3.13	0.14	0.74
	E2	0.51	-0.25	1.58
	E3	-1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	-2.81	-0.68	-0.41

Token chooses expert

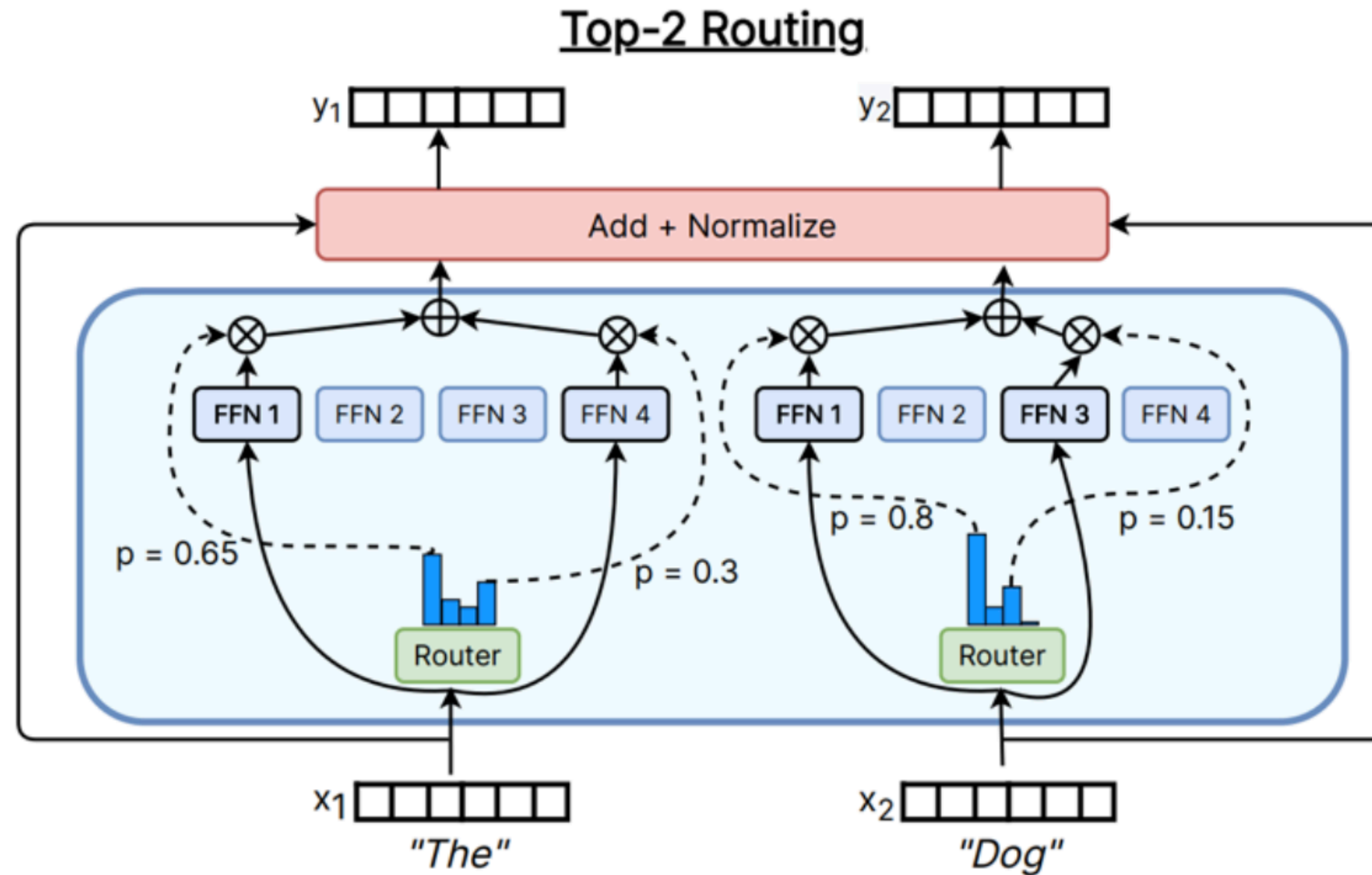
		Tokens		
		T1	T2	T3
Experts	E1	Choose Top-K		
	E2	0.51	-0.25	1.58
	E3	-1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	-2.81	-0.68	-0.41

Expert chooses token

		Tokens		
		T1	T2	T3
Experts	E1	3.13	0.14	0.74
	E2	0.51	-0.25	1.58
	E3	-1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	-2.81	-0.68	-0.41

Global routing via optimization

Top 2 Routing



Expert Selection and Output

- Multiply the input (x) by the router matrix (W)

$$H(x) = x * W$$

- Apply softmax to create probability distribution

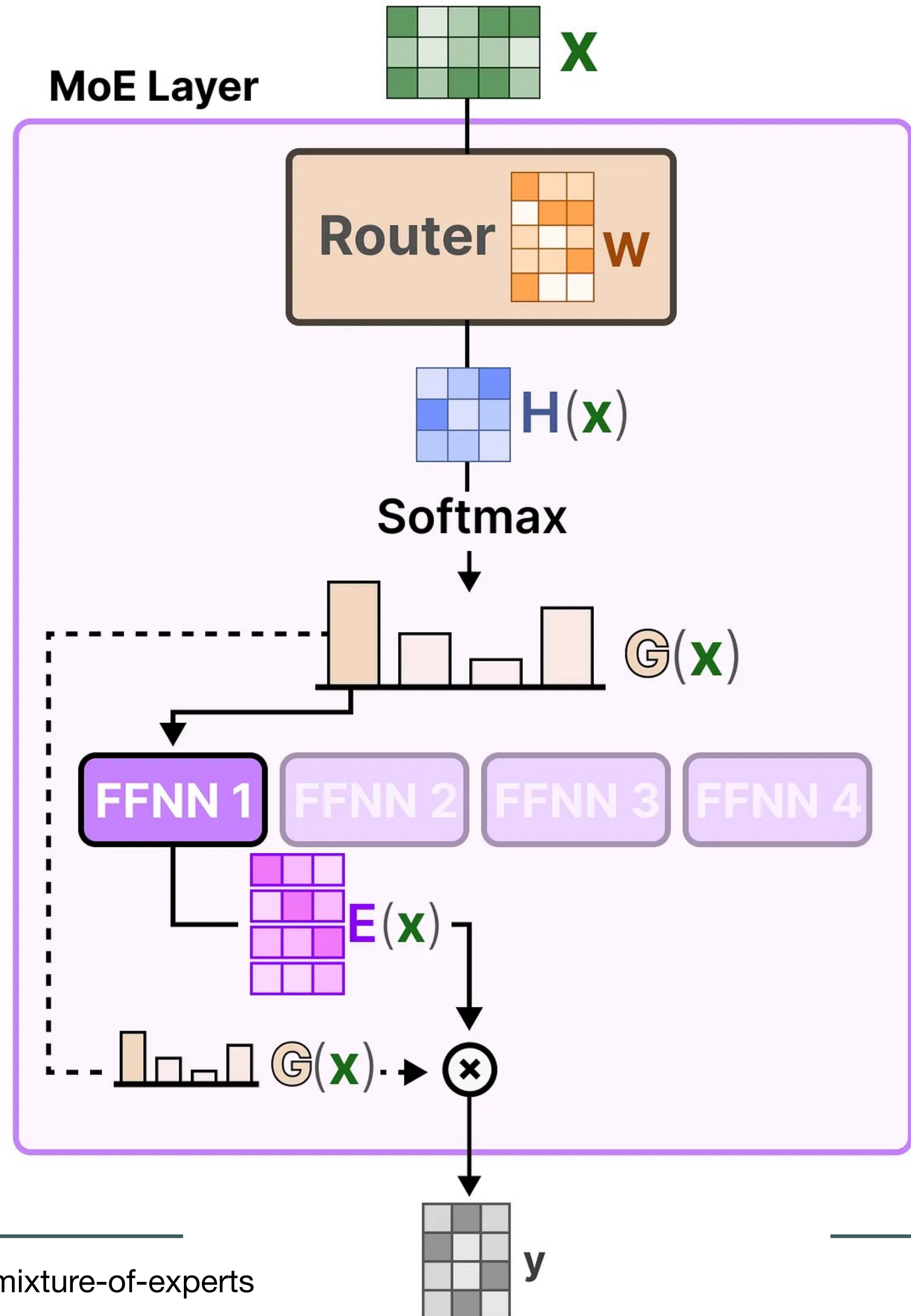
$$G(x) = \text{Softmax}(H(x))$$

- Multiply the output of each router with each selected expert and sum

$$y = \Sigma(G(x) * E(x))$$



Expert Selection and Output



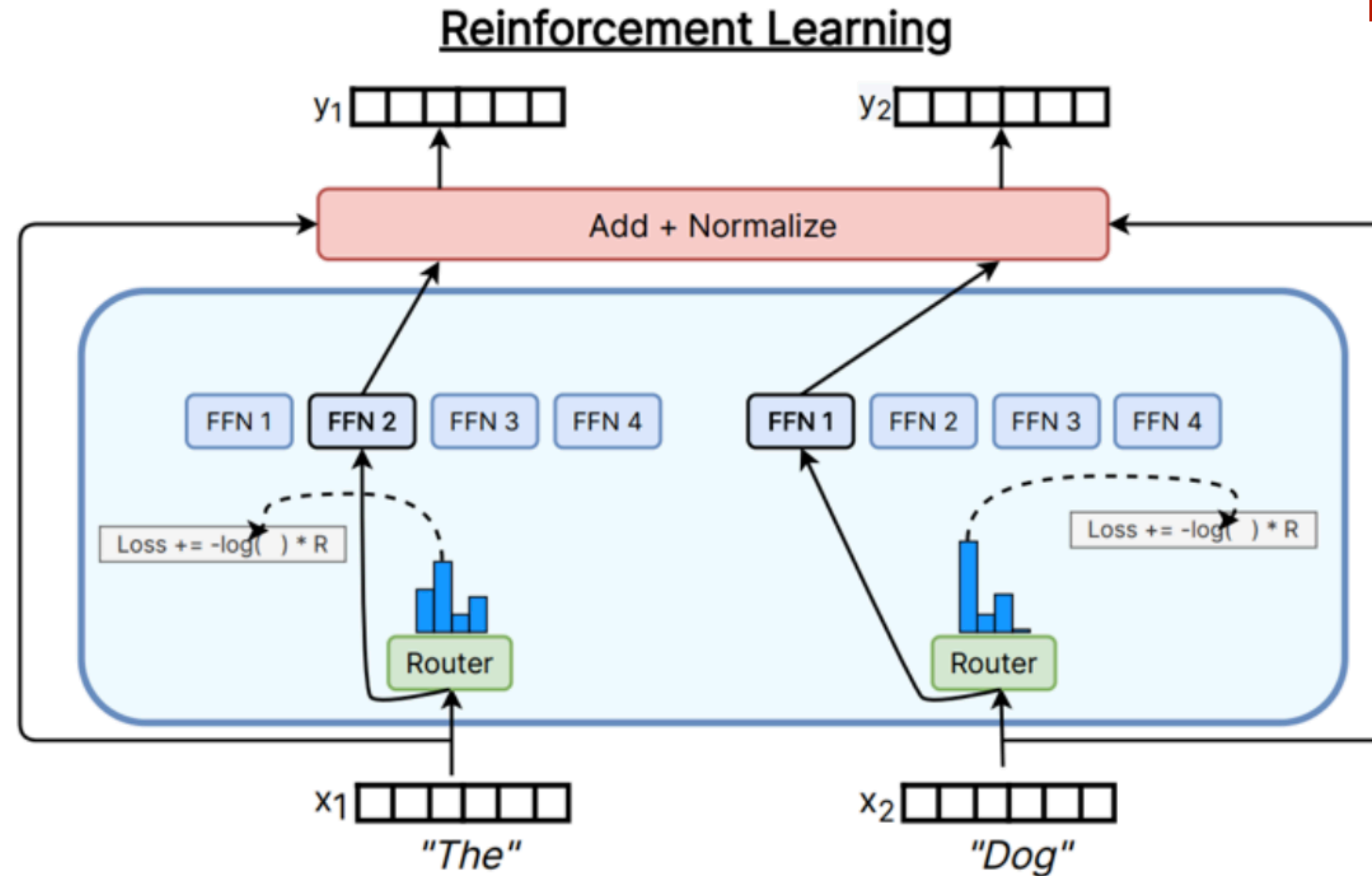
Models using Top k

- Switch Transformer - 1
- Gshard - 2
- Grok
- Mixtral
- Qwen
- DeepSeek



Routing/Gating With RL

Used in earlier works
not popular anymore



Routing Challenges

- Same expert(s) always chosen
 - Uneven distribution(load) on experts
 - Some experts are hardly trained
- Some experts train faster than others



Load Balancing

Load Balancing

- To achieve equal importance among experts during training and inference



Load Balancing

How?

- Add Gaussian Noise
- Token Choice Routing
- Load Balancing Loss (Auxiliary Loss)

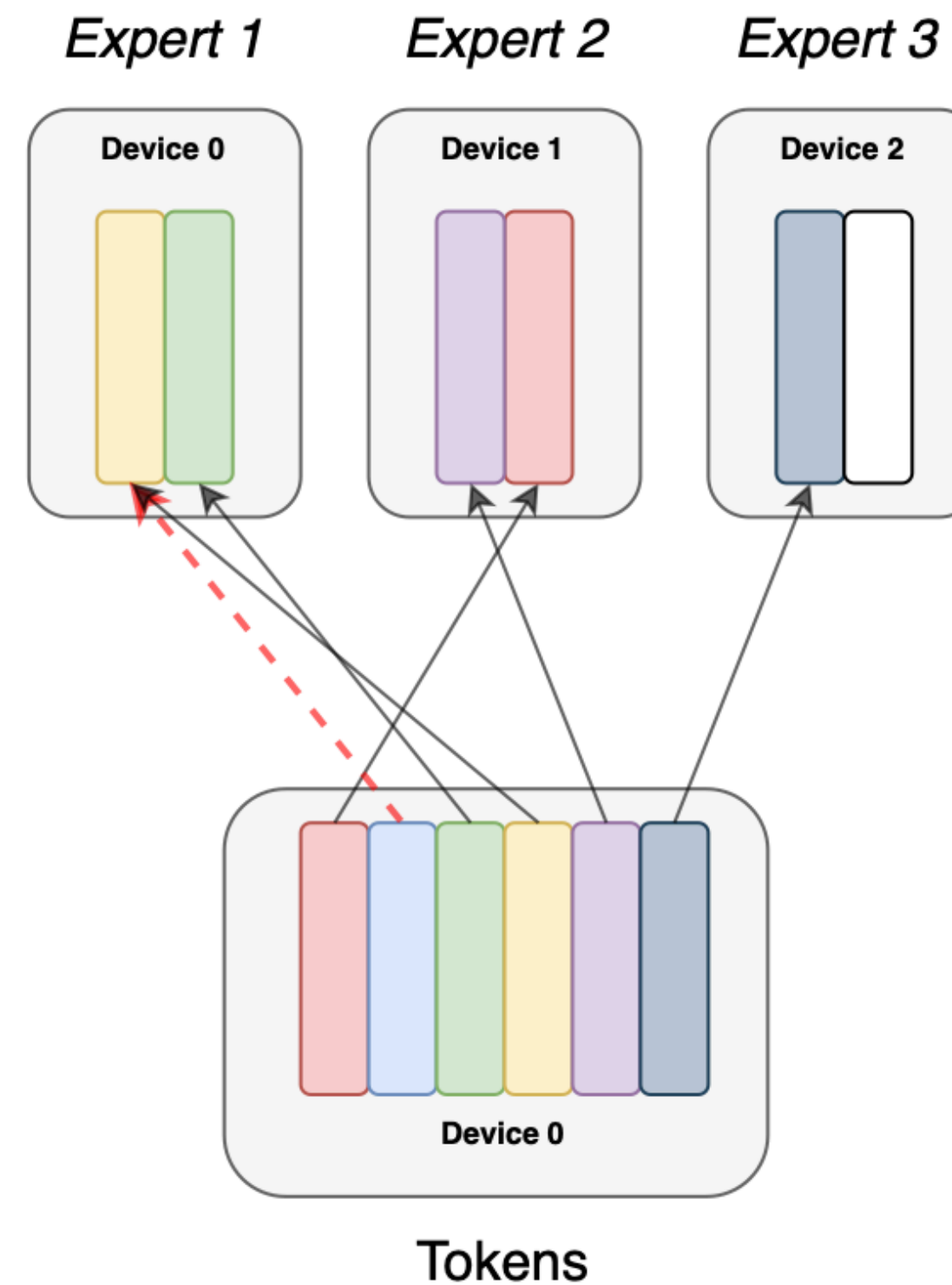


Expert Capacity

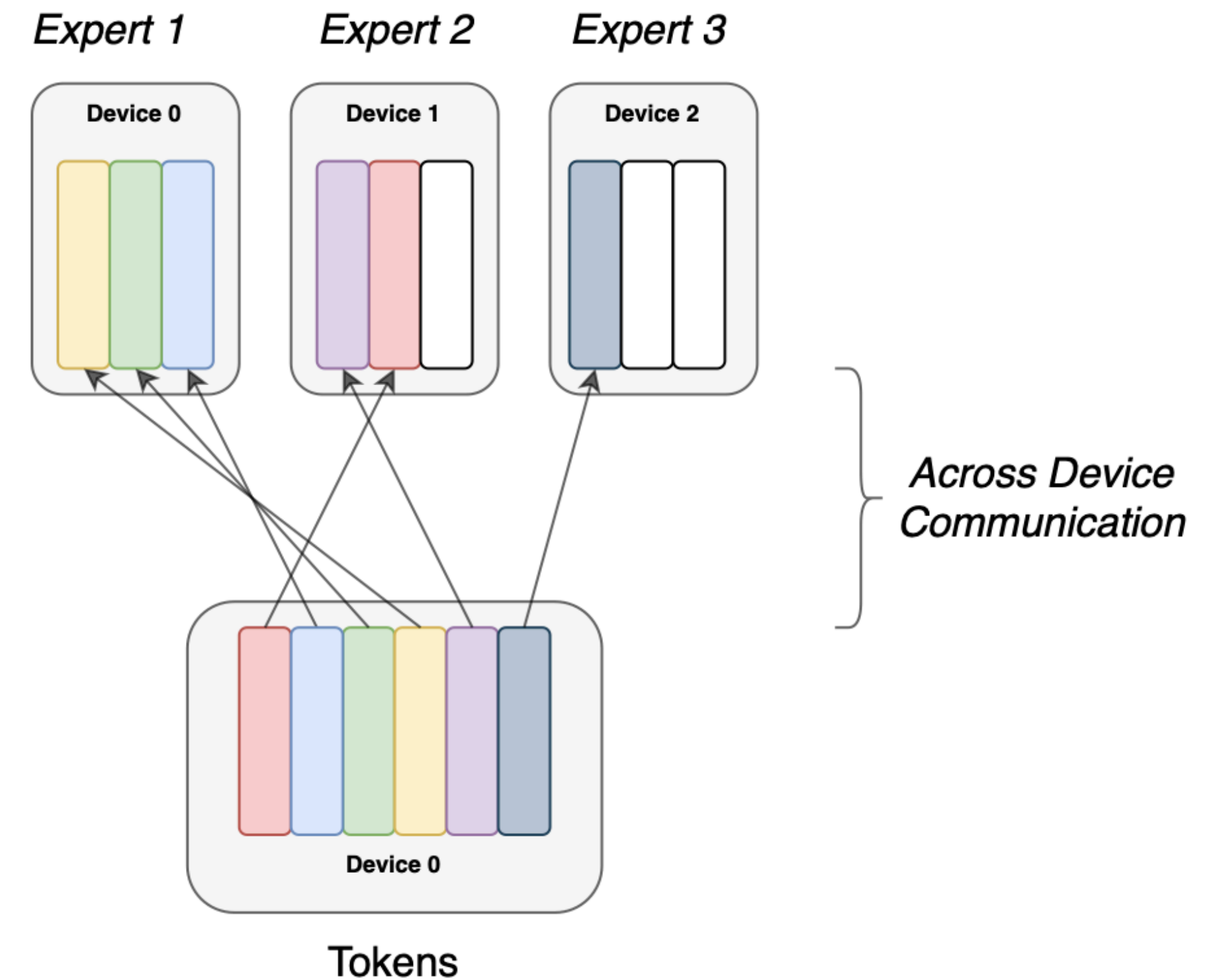
Terminology

- **Experts:** Split across devices, each having their own unique parameters. Perform standard feed-forward computation.
- **Expert Capacity:** Batch size of each expert. Calculated as
 - $(\text{tokens_per_batch} / \text{num_experts}) * \text{capacity_factor}$
- **Capacity Factor:** Used when calculating expert capacity. Expert capacity allows more buffer to help mitigate token overflow during routing.

(Capacity Factor: 1.0)



(Capacity Factor: 1.5)



Load Balancing Loss

- Sum the router values for each expert over the entire batch

$$\text{Importance Score per expert} = \Sigma(G(x))$$

- Calculate Coefficient of variation

$$CV = \sigma / \mu$$

- Auxiliary loss aims to lower the CV

$$\text{Load balancing Loss} = w * CV \text{ (w - constant scaling)}$$

Transformer Models over Years

Model	Layer	Activation
Original Base	FF	ReLU
GPT	FF	GeLU
mT5 - L	FF	GeGLU
GPT3		GeLU
LLaMA	FF	SwiGLU
Mistral	FF	SwiGLU
LLaMA 3.2	FF	SwiGLU
DeepSeek V3	MoE	SwiGLU
Qwen 3	MoE	SwiGLU
Kimi K2	MoE	SwiGLU

Transformer Models over Years - 2026

Model	NN	Num Experts	Activation	Params	Active Params
Qwen 3.5	MoE	512		397B	17B
Sarvam	MoE	128		105B	10.3B
Nemotron 3 Ultra	Mamba + Latent MoE			550B	55B
Mistral 4	MoE	128		119B	6B
Gemma 4	FFNN	-		12B	-
Qwen 3.6	MoE	256		35B	3B
DeepSeek V4	MoE	384	SwiGLU	1.6T	49B
Kimi k3	FFNN + LatentMoE	896	SiTU-GLU	2.8T	104B